Unidad 4

Bases de datos objeto-relacionales y orientadas a objetos

Tarea AD04

BD Objeto-Relacionales y Orientada a Objetos

Contenido

EJERCICIO 1. 2

1.1. Dar de alta en la base de datos nuevos concesionarios. Para dar de alta un nuevo concesionario la información necesaria es: CIF, nombre, dirección, provincia, teléfono y número de trabadores que trabajan en él. 3

1.2. Eliminar concesionarios de la base de datos (un concesionario sólo se podrá eliminar si no tiene ningún coche a la venta). 13

1.3. Dar de alta en la base de datos nuevos coches. Para dar de alta un nuevo coche la información necesaria es: matrícula, marca, modelo, kilómetros, precio y en que concesionario está (el concesionario debe existir en la base de datos). 16

1.4. Eliminar coches de la base de datos. 20

1.5. Mostrar todos los concesionarios ordenados por nombre. 23

1.6. Mostrar todos los coches de un concesionario indicado por el usuario, ordenados por precio. 26

1.7. Mostrar todos los coches que tengan menos de un determinado número de kilómetros (el número de kilómetros lo debe facilitar el usuario). 29

1.8. Rebajar el precio en un 10% de todos los coches de una determinada marca (la marca la debe introducir el usuario). 32

Descripción de la tarea

Caso práctico

Tu empresa ha sido contratada por una red de concesionarios de coches de segunda mano. Esta red quiere mantener centralizada la información de los coches que tienen a la venta en los diferentes concesionarios y lo van a hacer usando db40, que es una base de datos orientada a objetos. Tú formas parte del proyecto que se va a encargar de realizar esta tarea, por lo que junto con tus compañeros deberás crear la aplicación solicitada.

EJERCICIO 1.

Debes crear una aplicación con interfaz gráfica que gestione la información de una red de concesionarios de segunda mano. Los datos que maneja la aplicación estarán en una base de datos orientada a objetos usando db4o. Los objetos que manejará esta base de datos son concesionarios y coches.

Es necesario que la aplicación sea intuitiva y fácil de manejar, mostrando mensajes de control adecuados para el usuario. Por ejemplo, informando de cuándo se abre la base de datos, cuándo se cierra, del tipo de datos necesario para una acción, o de los posibles errores que se produzcan.

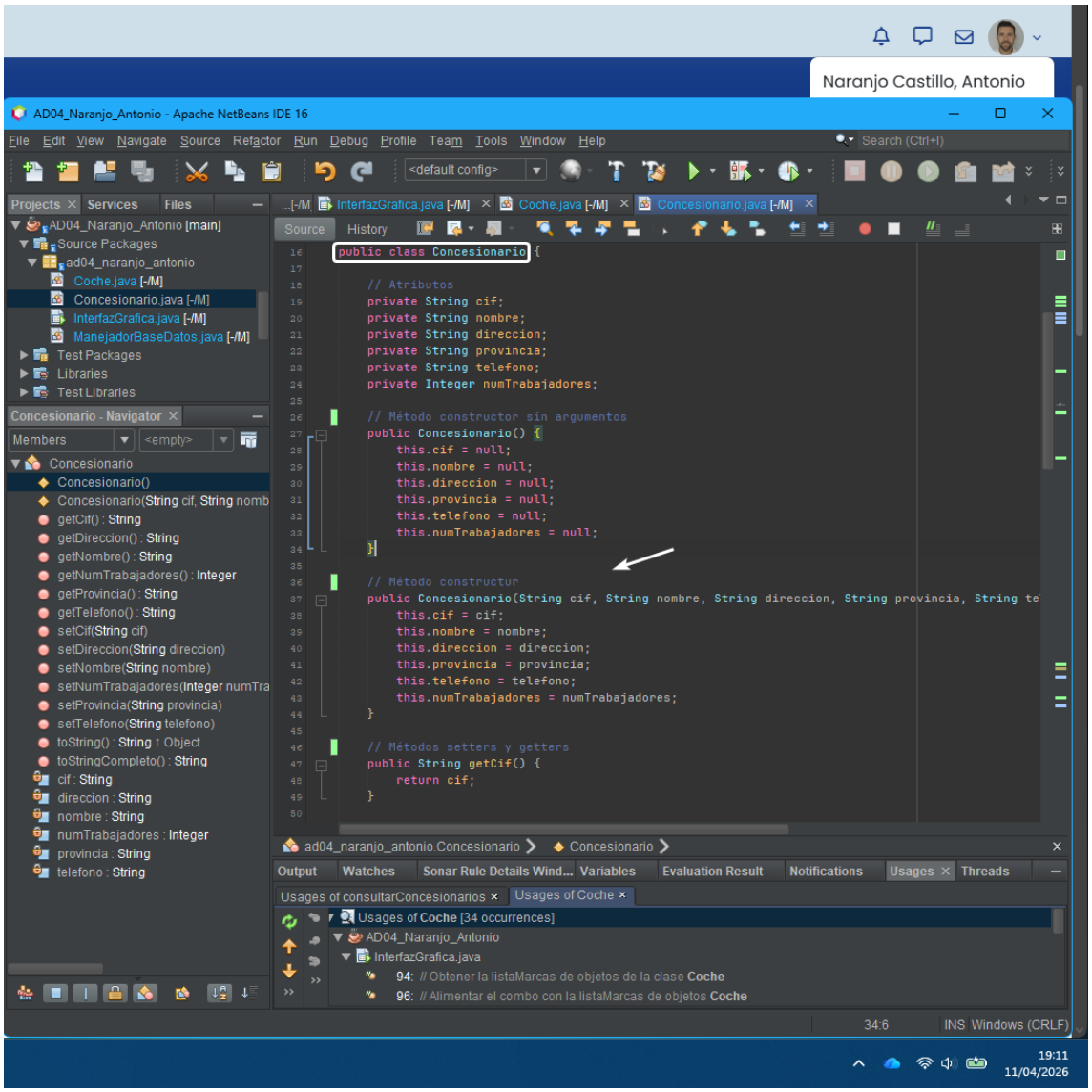
NOTA IMPORTANTE: Es necesario realizar una documentación explicando detalladamente toda la aplicación y realizando las pruebas de funcionamiento de todos los apartados anteriores.

La aplicación deberá permitir realizar la siguiente serie de acciones:

1.1. Dar de alta en la base de datos nuevos concesionarios. Para dar de alta un nuevo concesionario la información necesaria es: CIF, nombre, dirección, provincia, teléfono y número de trabadores que trabajan en él.

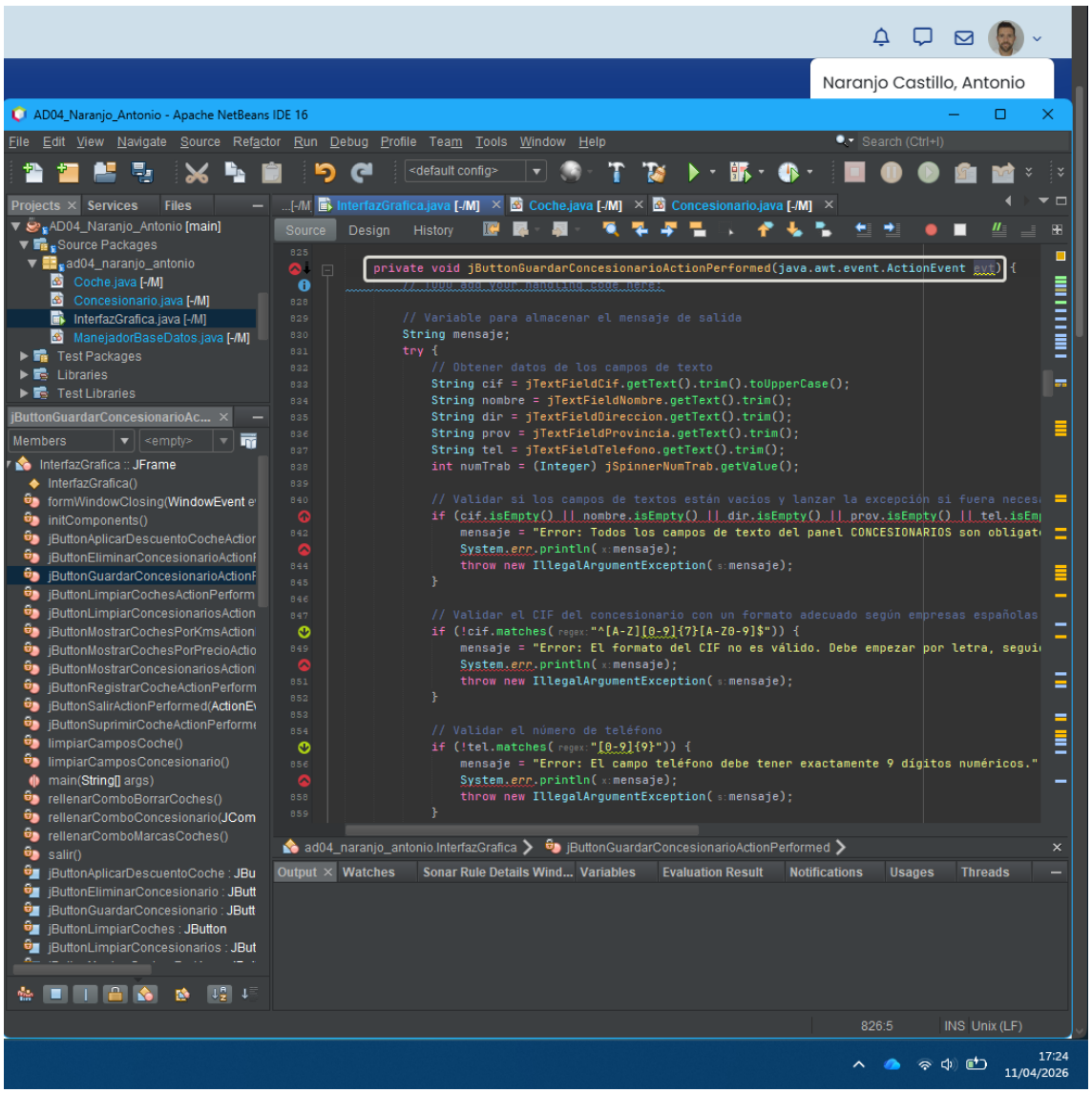
En primer lugar, se implementa la clase Concesionario, sus instancias son creadas durante los procesos de alta de datos o recuperadas desde el motor de persistencia mediante consultas. Definen la estructura de datos y el comportamiento de la entidad concesionario, permitiendo el encapsulamiento de sus atributos y la representación textual de los mismos en la interfaz gráfica.

Esta clase actúa como el modelo de datos para representar a los concesionarios en la aplicación. Define un conjunto de atributos privados que cubren la información necesaria: cif, nombre, direccion, provincia, telefono y numTrabajadores. La clase incluye un constructor sin argumentos (necesario para ciertos marcos de trabajo de persistencia) y un constructor parametrizado que facilita la creación de nuevas instancias con todos sus datos. La integridad de los atributos se gestiona a través de métodos de acceso y modificación (getters y setters).



Un aspecto diferencial de esta clase es la sobreescritura de métodos de visualización. El método @Override toString() ha sido diseñado específicamente para alimentar los componentes jComboBox, devolviendo una cadena formateada con el nombre y el CIF. Complementariamente, se incluye el método toStringCompleto(), el cual utiliza String.format() para generar una representación tabular y detallada de todos los atributos de la instancia, facilitando su volcado en consolas o áreas de texto de la interfaz de forma alineada.

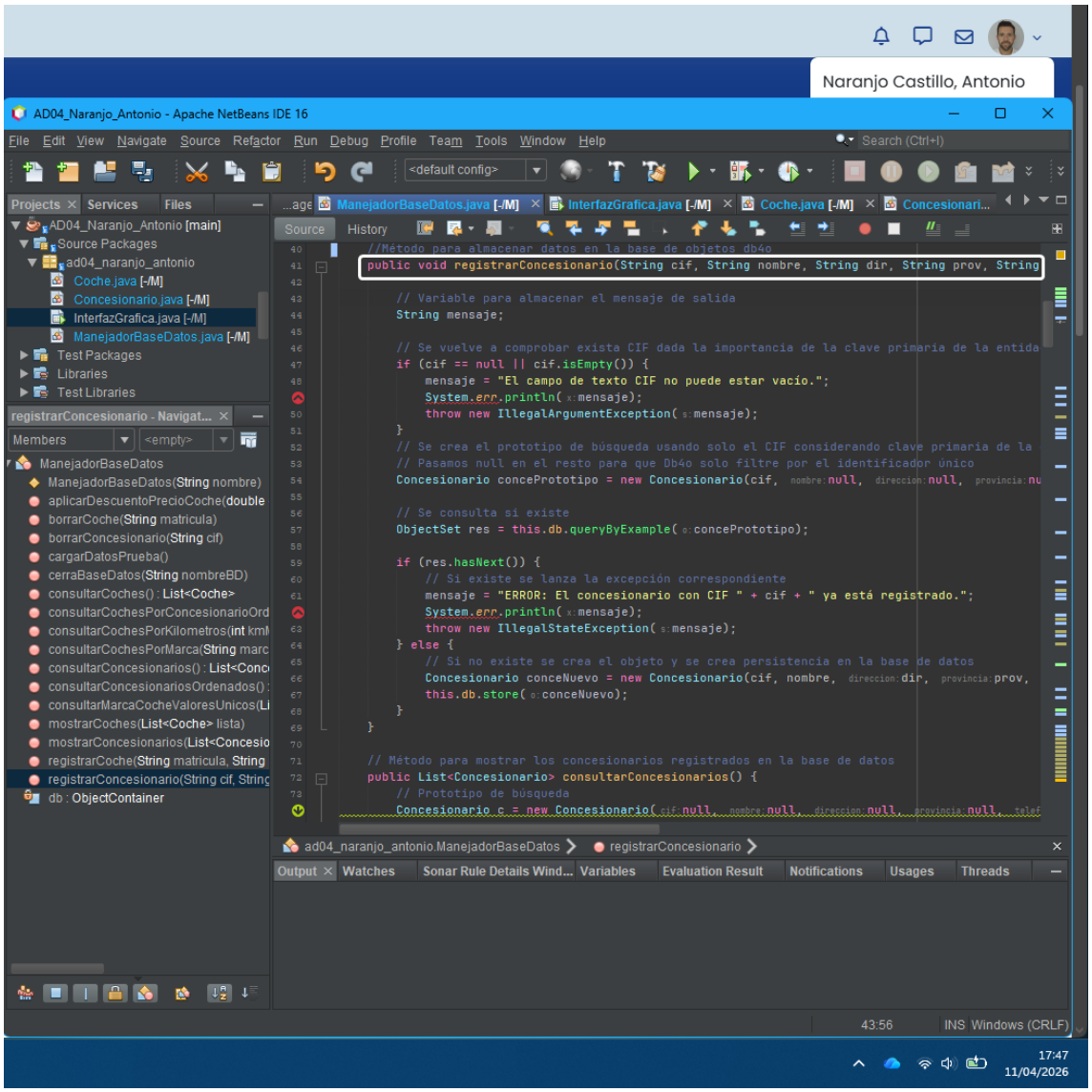
Para dar de alta nuevos concesionarios en la base de datos db4o se ejecuta el método jButtonGuardarConcesionarioActionPerformed de la clase InterfazGrafica, pulsando sobre el botón "Guardar" del panel Concesionarios. Su función es validar la entrada de datos del usuario, registrar un nuevo objeto Concesionario en la base de datos y actualizar los componentes visuales de la interfaz:



Este método gestiona el evento de pulsación del botón para dar de alta un nuevo concesionario en el sistema. Su funcionamiento se divide en tres fases críticas:

recolección, validación y persistencia. En primer lugar, se extraen los datos de la interfaz gráfica (campos de texto y spinner), aplicando un tratamiento preventivo mediante el método trim() para eliminar espacios y toUpperCase() para normalizar el CIF. Posteriormente, se realiza una validación exhaustiva que asegura que ningún campo obligatorio esté vacío y que tanto el CIF como el Teléfono cumplan con formatos específicos mediante expresiones regulares.

Si los datos son válidos, se delega la persistencia en el método registrarConcesionario de la clase ManejadorBaseDatos (encargado de la inserción en db4o). Tras un registro exitoso, el flujo actualiza la consistencia de la vista llamando repetidamente al método rellenarComboConcesionario para sincronizar los diferentes componentes jComboBox de la aplicación y, finalmente, ejecuta limpiarCamposConcesionario para resetear el formulario. El bloque cuenta con una estructura try-catch que captura excepciones específicas, diferenciando errores de formato (IllegalArgumentException) de intentos de duplicidad de datos en la base de objetos (IllegalStateException).



El método registrarConcesionario es invocado internamente por la interfaz gráfica tras superar las validaciones iniciales de los formularios, su función es asegurar la integridad de los datos mediante la comprobación de unicidad del CIF (clave primaria del objeto en cuestión) y persistir el nuevo objeto Concesionario en la base de datos orientada a objetos.

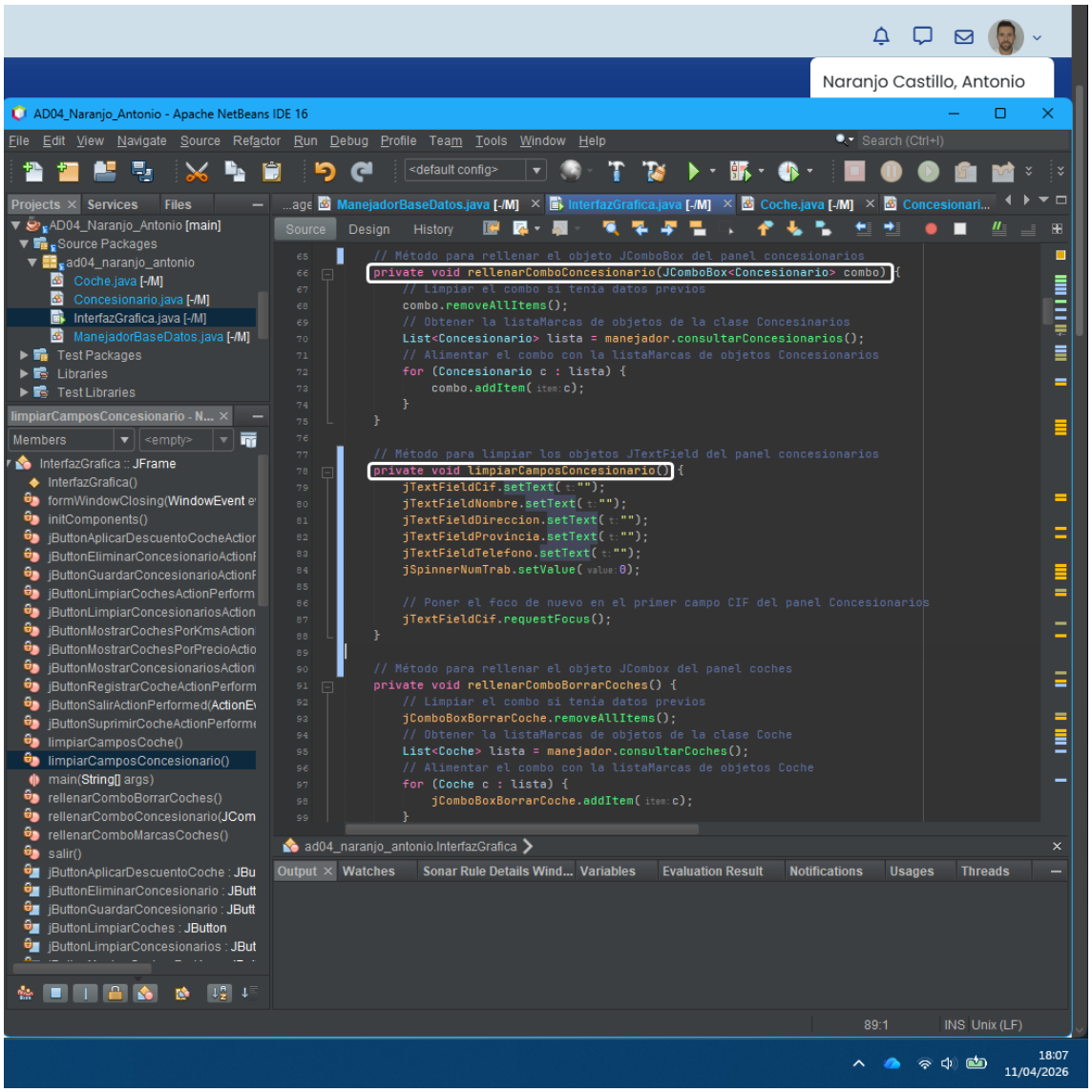
Este método implementa la lógica de persistencia para el alta de concesionarios, actuando como un filtro de integridad antes de la escritura en disco. En primera instancia, realiza una comprobación de seguridad sobre el parámetro cif para evitar entradas nulas o vacías. Dado que db4o no gestiona claves primarias de forma nativa como el modelo relacional, el método emplea la técnica Query by Example (QBE); para ello, instancia un objeto concePrototipo utilizando únicamente el CIF y dejando el resto de los atributos como null.

Al ejecutar db.queryByExample(concePrototipo), se le pregunta a la base de datos sobre la existencia previa de dicho identificador. Si el ObjectSet resultante contiene algún registro (res.hasNext()), se lanza una excepción de tipo IllegalStateException para evitar duplicidades. En caso contrario, se procede a la instanciación definitiva del objeto Concesionario con todos los parámetros recibidos y se consolida su persistencia mediante el método store() del objeto contenedor db.

El método rellenarComboConcesionario es invocado automáticamente tras realizar operaciones de escritura (guardado) o borrado, y durante la inicialización de la interfaz. Su función es sincronizar el contenido de un componente JComboBox específico con los registros actualizados de la clase Concesionario almacenados en la base de objetos.

Este método tiene como objetivo la actualización dinámica de los componentes de selección de la interfaz. El proceso se inicia ejecutando removeAllItems() sobre el objeto JComboBox recibido como parámetro, garantizando así la eliminación de residuos de datos previos y evitando duplicidades visuales. A continuación, el flujo recupera la colección actualizada de objetos desde la base de datos mediante la llamada al método consultarConcesionarios() de la clase ManejadorBaseDatos.

Una vez obtenida la estructura List<Concesionario>, se emplea un bucle for-each para recorrer cada instancia y añadirla al componente mediante el método addItem(c). Es importante destacar que, al estar el combo parametrizado con la clase de la entidad, este mostrará el valor devuelto por el método toString() de cada objeto Concesionario. Dado que se trata de diseño genérico, el método puede ser reutilizado para actualizar múltiples selectores de la aplicación (alta de coches, bajas o consultas) con una sola llamada.



Sobre el método limpiarCamposConcesionario, es invocado tras un registro exitoso de un concesionario o cuando se desea cancelar la edición actual para limpiar el formulario. Su función es resetear los valores de todos los componentes de entrada del panel de concesionarios y posicionar el cursor para una nueva entrada de datos.

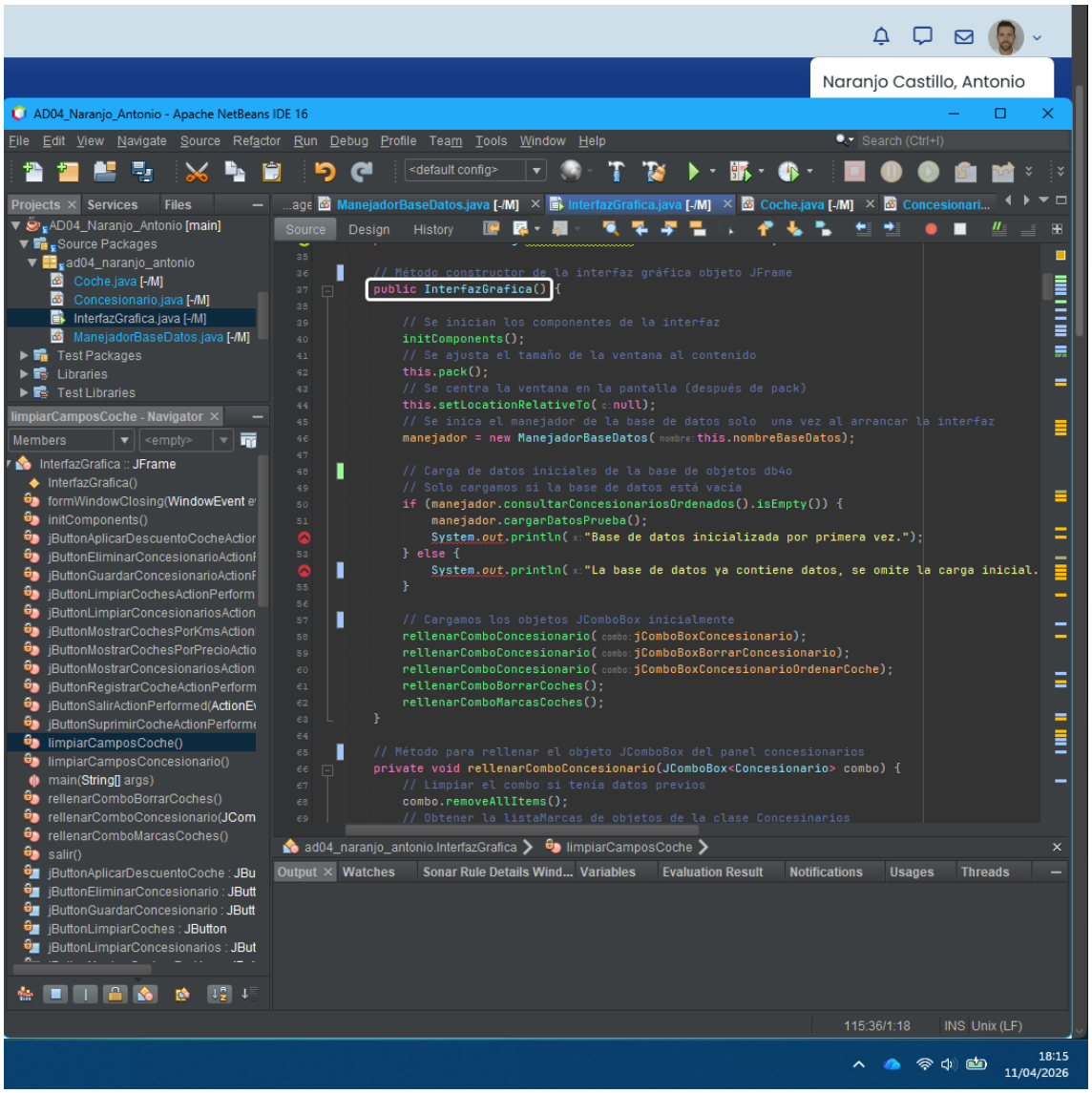
Este método realiza labores de mantenimiento y limpieza sobre la interfaz gráfica del panel de gestión de concesionarios. Su ejecución consiste en el vaciado sistemático de las cadenas de texto en los componentes jTextFieldCif, jTextFieldNombre, jTextFieldDireccion, jTextFieldProvincia y jTextFieldTelefono mediante el método setText(""). Asimismo, restablece el valor numérico del componente jSpinnerNumTrab a su estado inicial (cero) a través de setValue(0).

Para optimizar la usabilidad de la aplicación, el método utiliza la instrucción requestFocus() sobre el campo jTextFieldCif. Esta acción devuelve el foco del teclado al primer campo del formulario, permitiendo al usuario comenzar a escribir los datos de

un nuevo concesionario inmediatamente sin necesidad de utilizar el ratón, lo que agiliza significativamente el flujo de trabajo de introducción de datos.

Cabe destacar que existen métodos similares a los anteriores para realizar labores similares sobre el panel Coches, son los métodos rellenarComboBorrarCoches, rellenarComboMarcasCoches y limpiarCamposCoche.

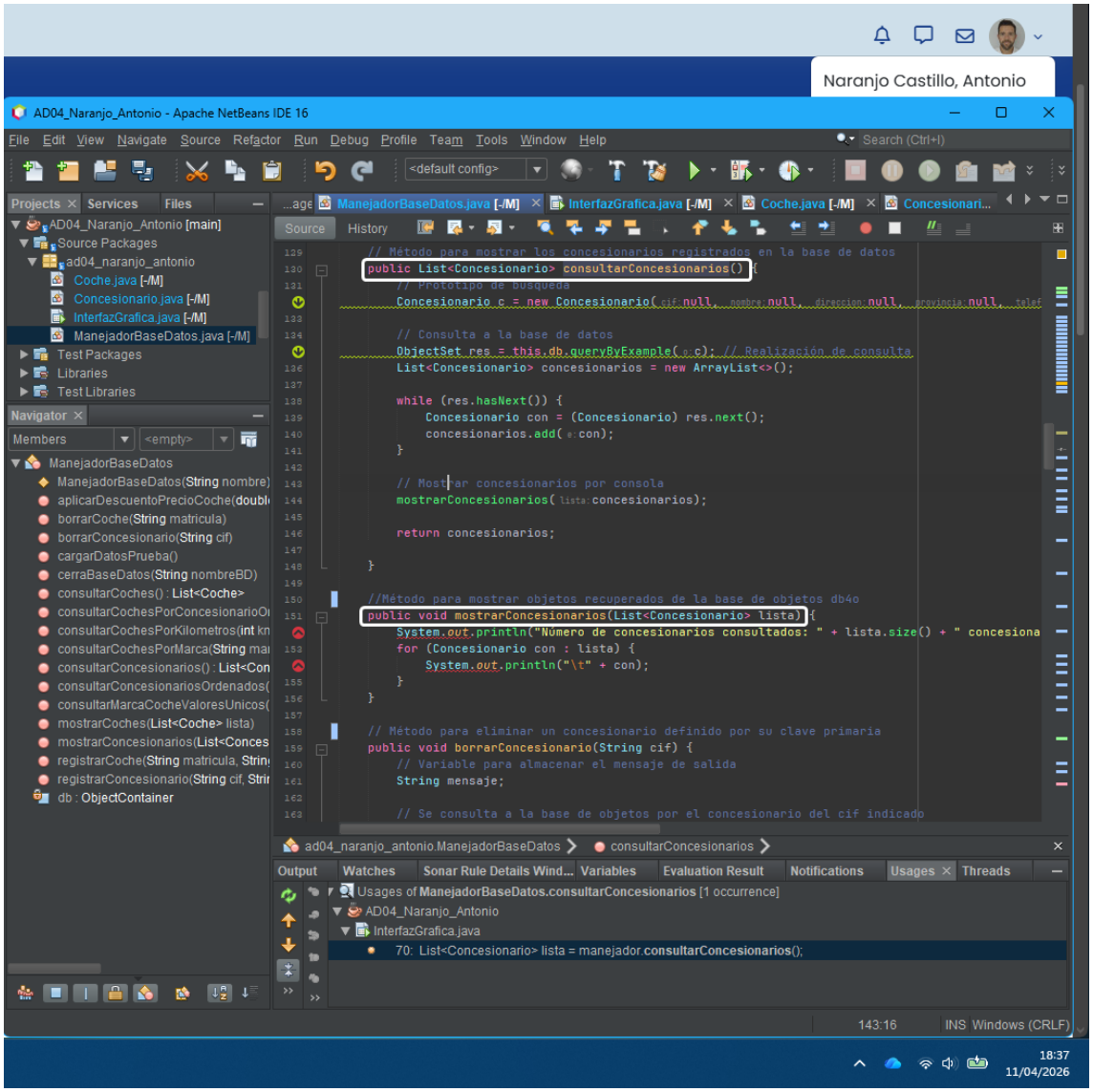
Sobre el inicio de la interfaz, el método constructor de la clase InterfazGrafica, se ejecuta de forma automática al instanciar la clase principal, generalmente en el método main. Su función es inicializar los componentes visuales, establecer la configuración de la ventana, conectar con el motor de base de datos db4o y asegurar que la interfaz muestre la información actualizada desde el primer instante.



Este método constructor lleva a cabo el arranque completo de la aplicación. En la fase de inicialización visual, invoca a initComponents() para cargar los elementos de Swing y emplea pack() y setLocationRelativeTo(null) para ajustar las dimensiones de la

ventana y centrarla automáticamente en el monitor del usuario. A nivel de lógica de negocio, instancia el objeto manejador de la clase ManejadorBaseDatos, estableciendo la conexión única con el archivo de la base de objetos mediante la propiedad nombreBaseDatos.

Este constructor constituye la lógica para alimentar todos los contenedores necesarios, para ello, mediante el método consultarConcesionariosOrdenados() realiza una consulta, tal que, si el resultado está vacío, ejecuta cargarDatosPrueba() para garantizar que la aplicación no se inicie sin contenido. Finalmente, el método asegura la consistencia de la interfaz llamando a múltiples rutinas de refresco, como rellenarComboConcesionario para los diferentes desplegables de concesionarios, así como rellenarComboBorrarCoches y rellenarComboMarcasCoches, permitiendo que el usuario interactúe con datos reales desde el primer momento.



El método consultarConcesionarios es invocado por la interfaz gráfica (a través del método rellenarComboConcesionario) cada vez que se necesita actualizar un listado o

un componente desplegable. Su misión es recuperar todos los objetos de la clase Concesionario almacenados en la base de datos db4o y devolverlos en una estructura de lista compatible con las colecciones de Java.

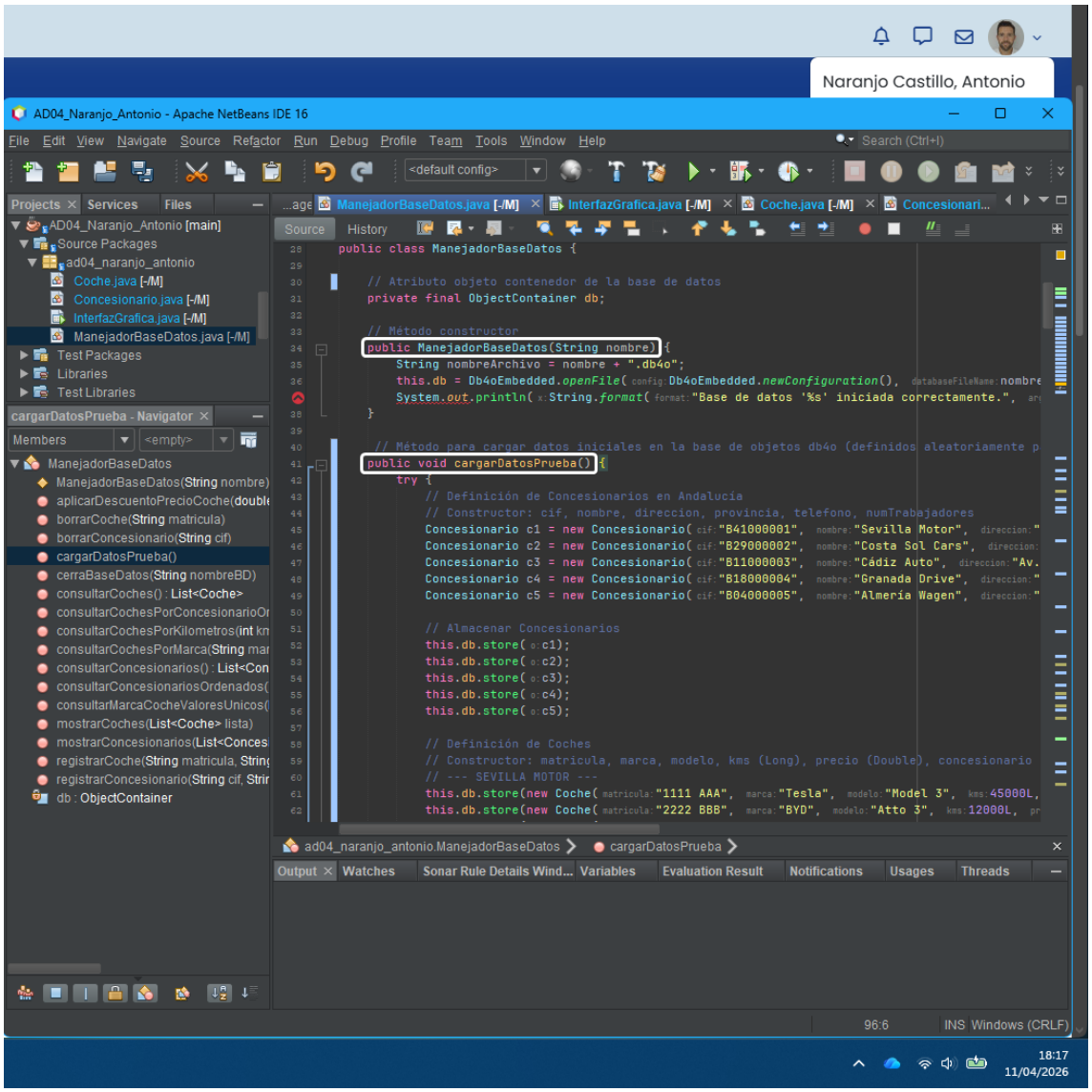
Este método implementa la lógica de recuperación de datos utilizando la técnica Query by Example (QBE). El proceso se inicia mediante la creación de un objeto "prototipo" de la clase Concesionario cuyos atributos se establecen como null; en el ecosistema de db4o, esto se interpreta como un comodín que indica que no se desea filtrar por ningún campo específico, devolviendo así todos los registros de dicha clase. Al ejecutar db.queryByExample(c), el motor devuelve un objeto ObjectSet, que actúa como un cursor sobre los resultados encontrados.

A continuación, el método inicializa un ArrayList y emplea un bucle while para iterar sobre el conjunto de resultados. En cada iteración, se utiliza res.next() para extraer el objeto, se realiza un casting explícito a la clase Concesionario y se añade a la lista final. Antes de volcar la colección, se llama al método interno mostrarConcesionarios pasándole dicha lista de objetos como argumento para volcar la información por la consola del sistema con fines de depuración o ayuda al desarrollador.

Sobre el constructor de la clase ManejadorBaseDatos y el método cargarDatosPrueba, el constructor se ejecuta al iniciar la aplicación, mientras que cargarDatosPrueba es invocado condicionalmente si la base de datos se detecta vacía durante el arranque de la interfaz gráfica. Su función es establecer la conexión física con el motor de base de objetos db4o y alimentar el sistema con una estructura inicial de objetos (Concesionario y Coche) planteadas a modo de ejemplo, para garantizar la operatividad inmediata del software.

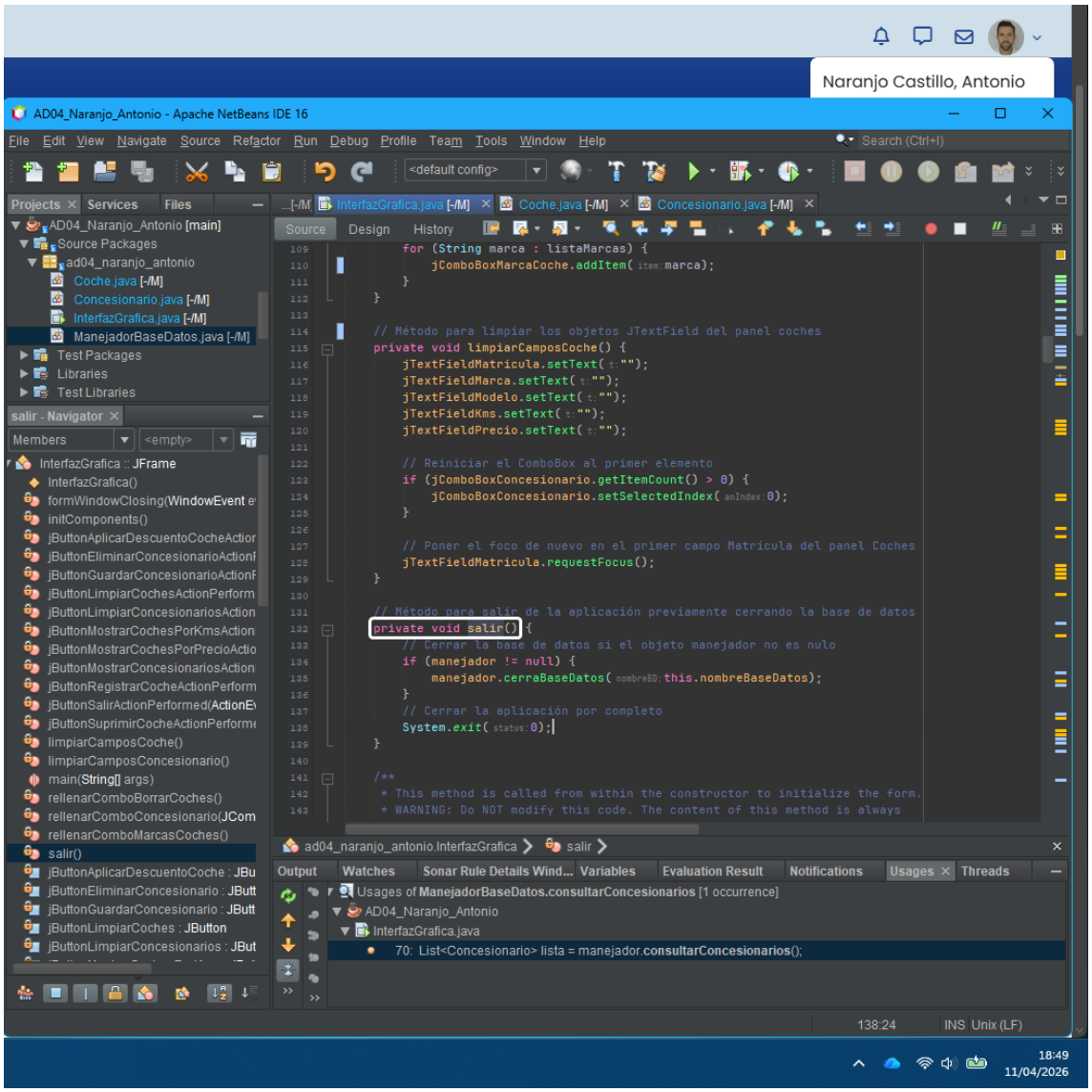
El constructor de esta clase es el responsable de abrir el contenedor de objetos mediante la instrucción Db4oEmbedded.openFile. Para ello, concatena el nombre recibido con la extensión .db4o y utiliza una configuración por defecto para inicializar la propiedad this.db, que actuará como el motor de persistencia durante toda la sesión. Por su parte, el método cargarDatosPrueba implementa la lógica de generación de datos iniciales. El proceso comienza instanciando objetos de la clase Concesionario con datos los cuales son persistidos inmediatamente mediante el método store().

Una vez creados los concesionarios, el método procede a instanciar y almacenar múltiples objetos de la clase Coche. Es relevante destacar cómo se mantiene la integridad referencial propia de las bases de datos orientadas a objetos: al crear un Coche, se le pasa como parámetro la instancia del objeto Concesionario previamente creado (por ejemplo, c1, c2, etc.), estableciendo un vínculo directo entre ambos objetos sin necesidad de claves ajenas numéricas. Todo el proceso está envuelto en un bloque try-catch para capturar cualquier error que se pudiera producir durante la fase inicial de escritura.



Sobre el método salir, es invocado normalmente al cerrar la ventana principal o al pulsar un botón específico de "Salir" en la interfaz. Su función es asegurar el cierre correcto del contenedor de objetos db4o para evitar la corrupción del archivo y finalizar la ejecución del proceso de la Java Virtual Machine (JVM).

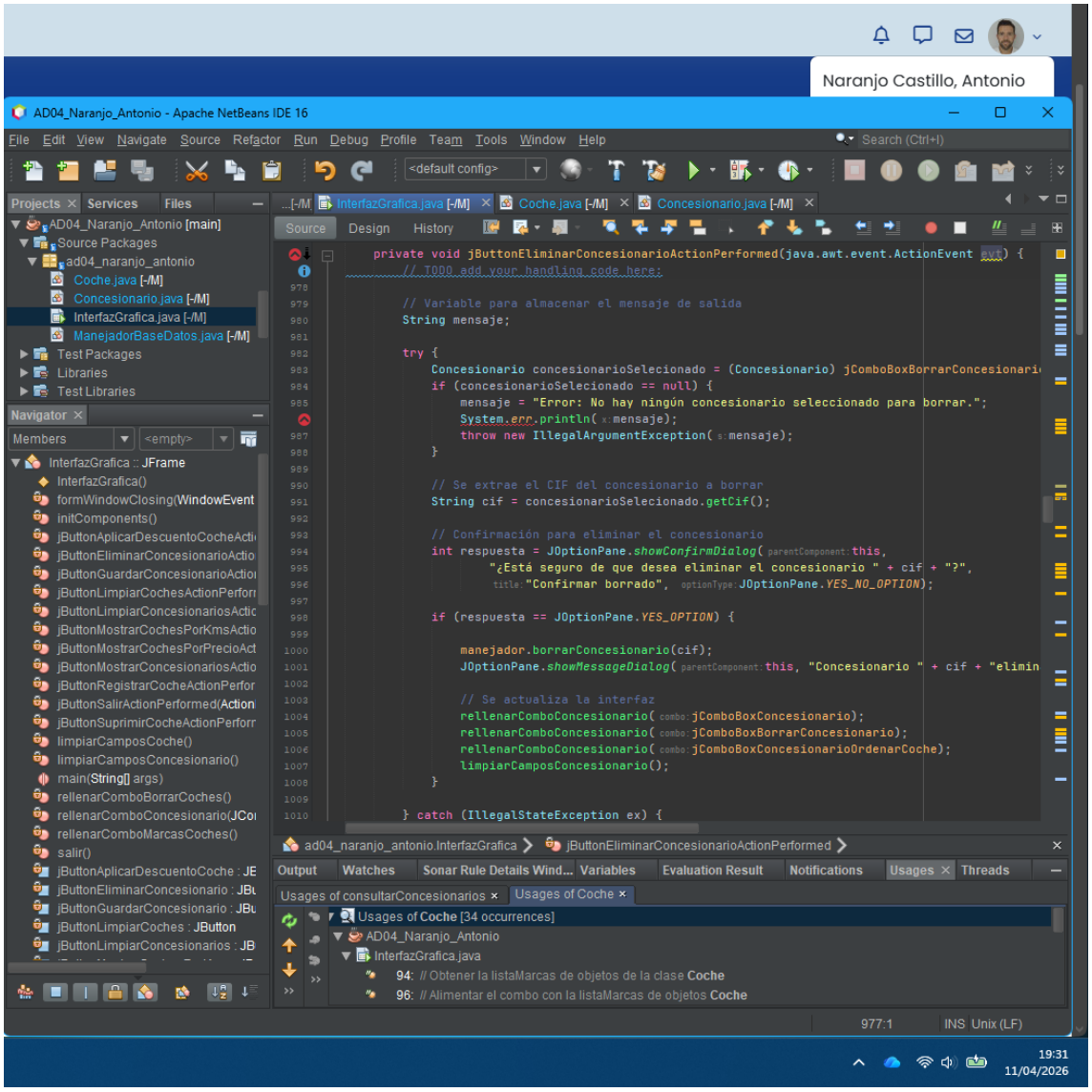
Este método implementa la rutina de cierre de la aplicación, priorizando la seguridad de la persistencia de datos. Antes de proceder al apagado del sistema, realiza una comprobación de seguridad verificando que el objeto manejador haya sido instanciado correctamente (manejador != null). Si la condición se cumple, se invoca al método cerraBaseDatos de la clase ManejadorBaseDatos, pasando como argumento la propiedad this.nombreBaseDatos; esta acción es crítica en db4o, ya que garantiza que todos los cambios pendientes en la memoria se vuelquen al disco y que el archivo .db4o se desbloquee correctamente. Finalmente, el método ejecuta la instrucción System.exit(0), la cual detiene de forma inmediata todos los hilos de ejecución de la interfaz gráfica y cierra la aplicación devolviendo un código de estado exitoso al sistema operativo.


Naranjo Castillo, Antonio
AD04_Naranjo_Antonio - Apache NetBeans IDE 16
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
Projects Services Files
AD04_Naranjo_Antonio [main]
Source Packages
ad04_naranjo_antonio
Coche.java [-/M]
Concesionario.java [-/M]
InterfazGrafica.java [-/M]
ManejadorBaseDatos.java [-/M]
Test Packages
Libraries
Test Libraries
salir - Navigator
Members
InterfazGrafica :: JFrame
InterfazGrafica()
formWindowClosing(WindowEvent e
initComponents()
jButtonAplicarDescuentoCocheActior
jButtonEliminarConcesionarioActionF
jButtonGuardarConcesionarioActionF
jButtonLimpiarCochesActionPerform
jButtonLimpiarConcesionariosAction
jButtonMostrarCochesPorKmsAction
jButtonMostrarCochesPorPrecioActio
jButtonMostrarConcesionariosAction
jButtonRegistrarCocheActionPerform
jButtonSalirActionPerformed(ActionE
jButtonSuprimirCocheActionPerform
limpiarCamposCoche()
limpiarCamposConcesionario()
main(String[] args)
rellenarComboBorrarCoches()
rellenarComboConcesionario(JCom
rellenarComboMarcasCoches()
salir()
jButtonAplicarDescuentoCoche : JBu
jButtonEliminarConcesionario : JButt
jButtonGuardarConcesionario : JButt
jButtonLimpiarCoches : JButton
jButtonLimpiarConcesionarios : JBut
InterfazGrafica.java [-/M]
Coche.java [-/M]
Concesionario.java [-/M]
Source Design History
for (String marca : listaMarcas) {
jComboBoxMarcaCoche.addItem(item: marca);
}
// Método para limpiar los objetos JTextField del panel coches
private void limpiarCamposCoche() {
jTextFieldMatricula.setText(t: "");
jTextFieldMarca.setText(t: "");
jTextFieldModelo.setText(t: "");
jTextFieldKms.setText(t: "");
jTextFieldPrecio.setText(t: "");
// Reiniciar el ComboBox al primer elemento
if (jComboBoxConcesionario.getItemCount() > 0) {
jComboBoxConcesionario.setSelectedIndex(anIndex: 0);
}
// Poner el foco de nuevo en el primer campo Matricula del panel Coches
jTextFieldMatricula.requestFocus();
}
// Método para salir de la aplicación previamente cerrando la base de datos
private void salir() {
// Cerrar la base de datos si el objeto manejador no es nulo
if (manejador != null) {
manejador.cerraBaseDatos(nombreBD: this.nombreBaseDatos);
}
// Cerrar la aplicación por completo
System.exit(status: 0);
}
/**
* This method is called from within the constructor to initialize the form.
* WARNING: Do NOT modify this code. The content of this method is always
ad04_naranjo_antonio.InterfazGrafica
salir
Output Watches Sonar Rule Details Wind... Variables Evaluation Result Notifications Usages Threads
Usages of ManejadorBaseDatos.consultarConcesionarios [1 occurrence]
AD04_Naranjo_Antonio
InterfazGrafica.java
70: List<Concesionario> lista = manejador.consultarConcesionarios();
138:24 INS Unix (LF)
18:49
11/04/2026

1.2. Eliminar concesionarios de la base de datos (un concesionario sólo se podrá eliminar si no tiene ningún coche a la venta).

Para eliminar un concesionario se pulsa el botón "Eliminar" dentro del panel Eliminar Objetos, ejecutándose el método jButtonEliminarConcesionarioActionPerformed, su función es gestionar el flujo de borrado de un concesionario seleccionado, solicitando confirmación al usuario y actualizando la interfaz tras la eliminación en la base de datos.

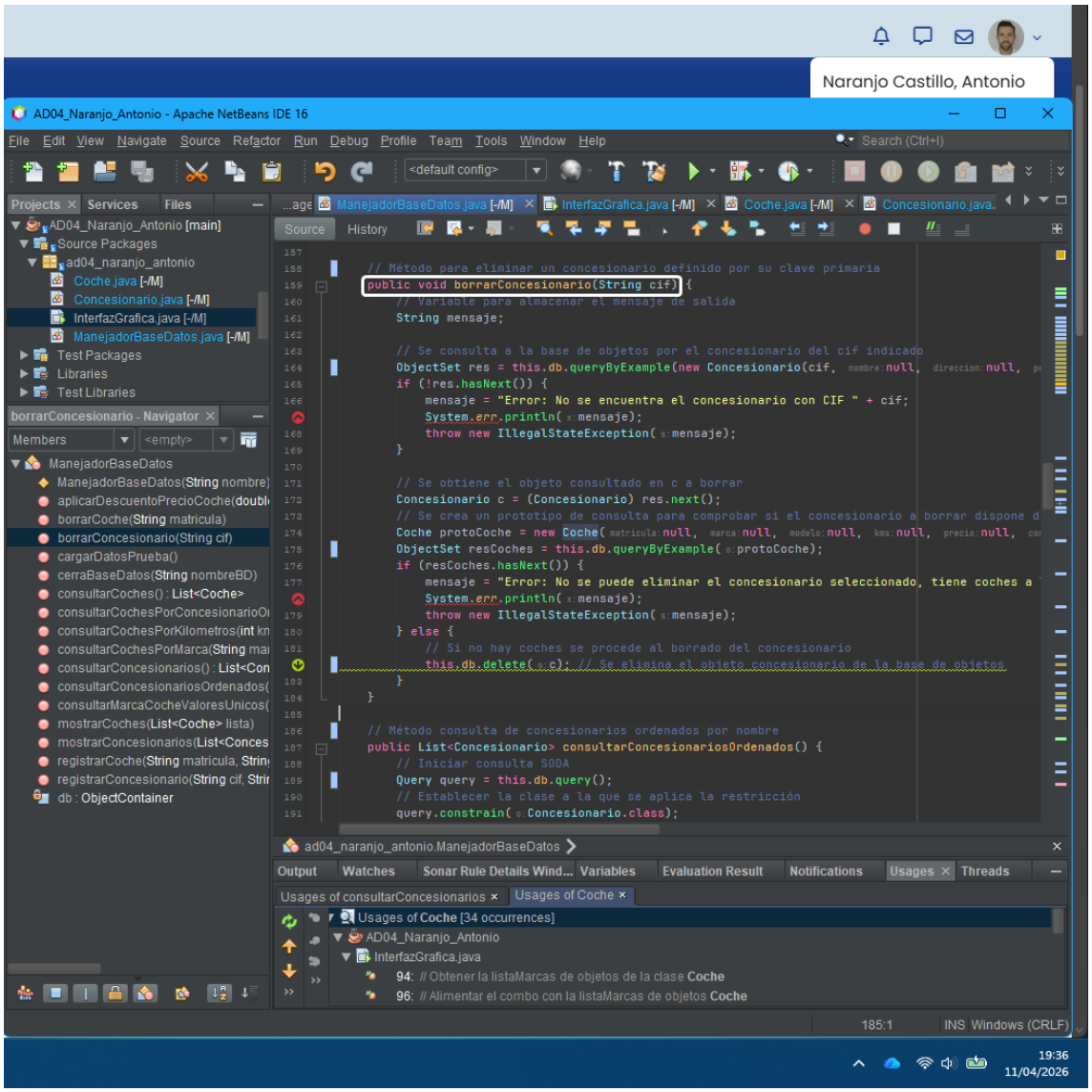
Este método controla el proceso de eliminación de un registro de concesionario. En primera instancia, recupera el objeto Concesionario seleccionado a través del componente jComboBoxBorrarConcesionario. Tras validar que existe una selección válida, extrae el atributo CIF para identificar la única entidad (CIF es clave primaria). Antes de proceder, el método lanza un cuadro de diálogo interactivo mediante JOptionPane.showConfirmDialog, lo que obliga al usuario a ratificar su intención de borrado mediante una respuesta afirmativa (YES_OPTION).



Una vez confirmada la acción, se delega la ejecución al método borrarConcesionario(cif) del objeto manejador. Tras un borrado exitoso, el flujo

garantiza la integridad visual de la aplicación invocando tres veces al método rellenarComboConcesionario para actualizar los selectores de los paneles de alta, baja y ordenación, además de ejecutar limpiarCamposConcesionario. El bloque try-catch es fundamental aquí, captura específicamente excepciones de tipo IllegalStateException, las cuales se disparan si el concesionario tiene coches asociados, impidiendo así un borrado que rompería la integridad referencial de la base de objetos.

El método borrarConcesionario, es invocado por la capa de interfaz gráfica tras la confirmación del usuario, recibiendo como parámetro el identificador único (CIF). Su misión es localizar el objeto Concesionario en la base de datos y proceder a su eliminación física mediante el método delete(), siempre que no existan dependencias jerárquicas con la entidad Coche.



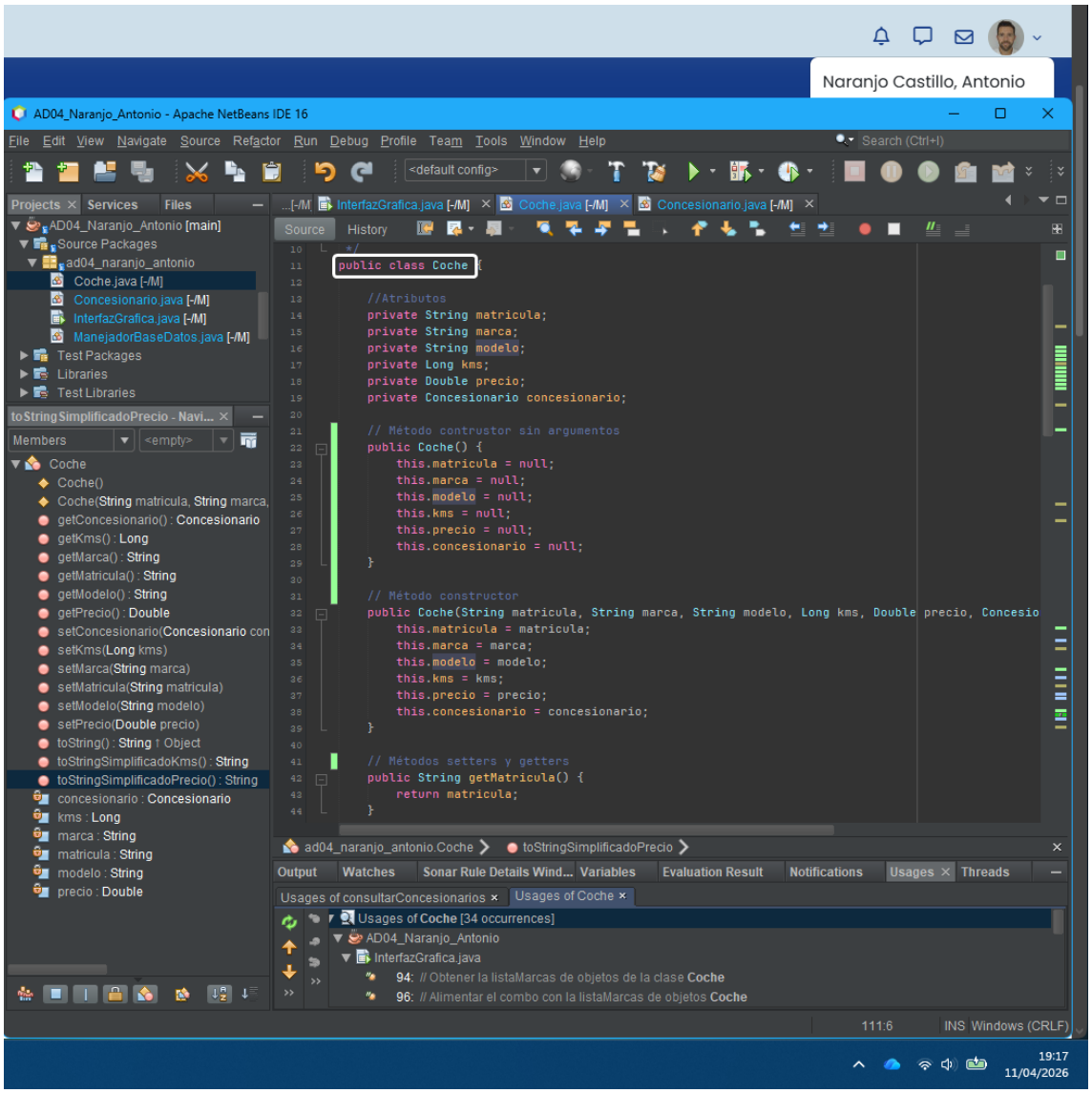
Este método constituye el bloque principal de la lógica de eliminación controlada de objetos en db4o. El proceso comienza con una búsqueda QBE para recuperar la instancia completa del concesionario a partir de su CIF. Si el ObjectSet está vacío, se

lanza una excepción informando de la inexistencia del registro. Una vez obtenida la instancia, el método realiza una validación de integridad referencial creando un prototipo de la clase Coche pasando el concesionario localizado como atributo de búsqueda.

Al ejecutar la consulta sobre este prototipo, se comprueba si existen vehículos vinculados a dicho centro. Si se detectan registros (resCoches.hasNext()), el flujo se interrumpe lanzando una excepción de tipo IllegalStateException, impidiendo así que queden "coches huérfanos" en la base de objetos. Solo en el caso de que el concesionario no posea coches asociados, se ejecuta la instrucción this.db.delete(c), eliminando definitivamente el objeto del contenedor de persistencia.

1.3. Dar de alta en la base de datos nuevos coches. Para dar de alta un nuevo coche la información necesaria es: matrícula, marca, modelo, kilómetros, precio y en que concesionario está (el concesionario debe existir en la base de datos).

Lo primero, implementar la Clase Coche, sus instancias se crean al registrar nuevos vehículos y se recuperan mediante consultas SODA o QBE para su visualización y gestión en la interfaz. De manera similar a la Clase Concesionario, definen la estructura y el comportamiento de la entidad coche, permitiendo el almacenamiento de sus atributos y estableciendo una relación directa con el objeto concesionario donde se encuentra ubicado.

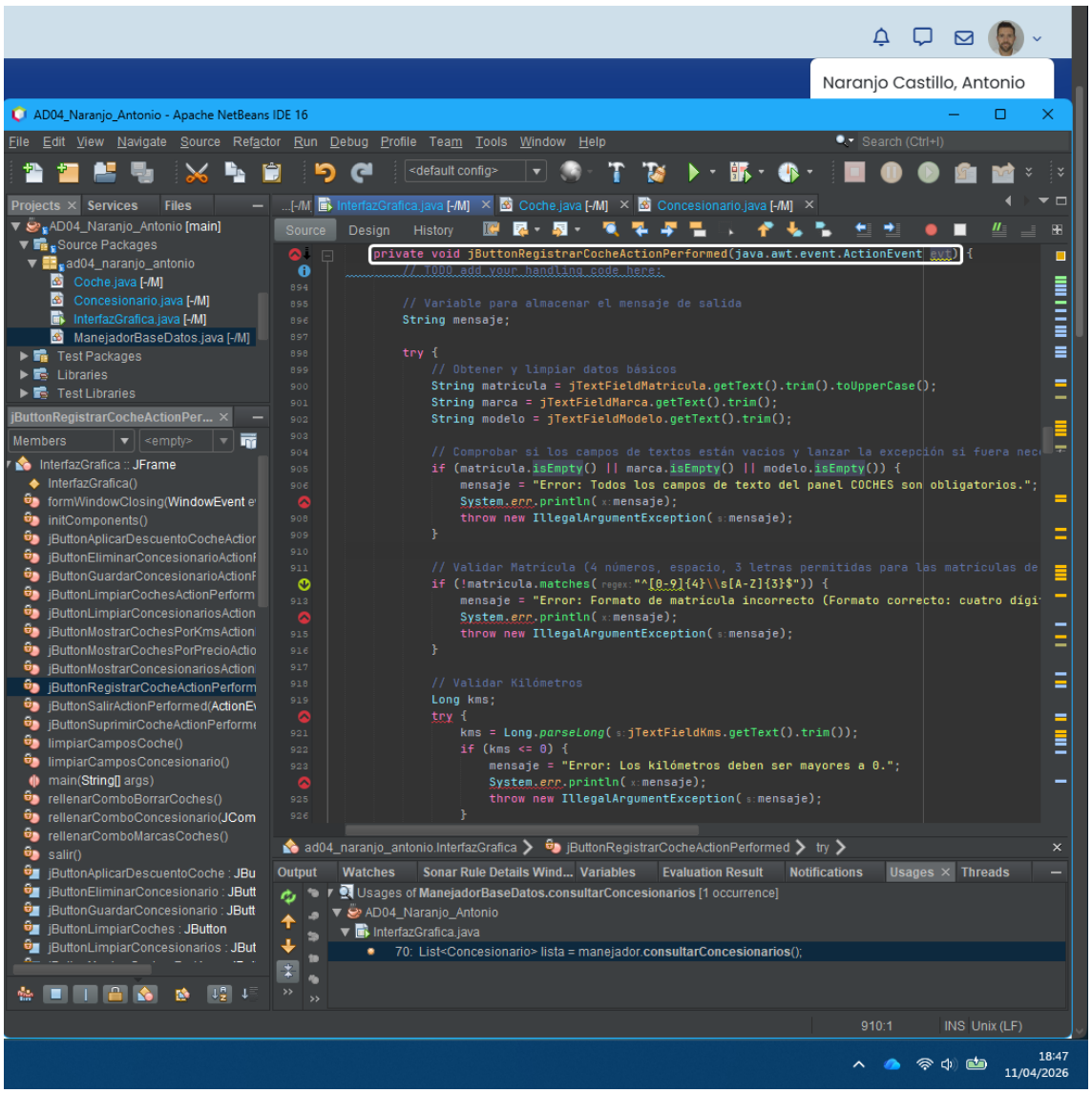


Esta clase constituye el núcleo de la información sobre los vehículos de la aplicación. Define atributos específicos como matricula, marca, modelo, kms y precio. Un detalle fundamental es la inclusión del atributo concesionario de tipo objeto, lo que permite a db4o gestionar una relación de asociación directa en lugar de utilizar simples identificadores numéricos. Se trata de un objeto estructurado a diferencia de Concesionario que era un objeto simple. La clase cumple con el estándar de

encapsulamiento al proporcionar constructores (por defecto y parametrizado) y métodos getter y setter para todas sus propiedades.

La clase destaca por ofrecer varias opciones para representar los datos mediante la sobreescritura del método @Override toString(), configurado para mostrar la marca, el modelo y la matrícula en componentes jComboBox. Además, implementa métodos de formato adicionales como toStringSimplificadoKms() y toStringSimplificadoPrecio(). Estos métodos utilizan String.format() para generar salidas tabuladas que facilitan la lectura de listados específicos (ordenados por kilometraje o precio), demostrando una integración limpia entre el modelo de datos y las necesidades de visualización de la capa de presentación.

A continuación, se ejecuta el método jButtonRegistrarCocheActionPerformed al pulsar el botón "Guardar" del panel de gestión de Vehículos (Coches). Lleva a cabo la validación integral y el formato de los datos del vehículo, recuperar la asociación con un objeto Concesionario existente y persistir el nuevo objeto Coche en la base de datos.



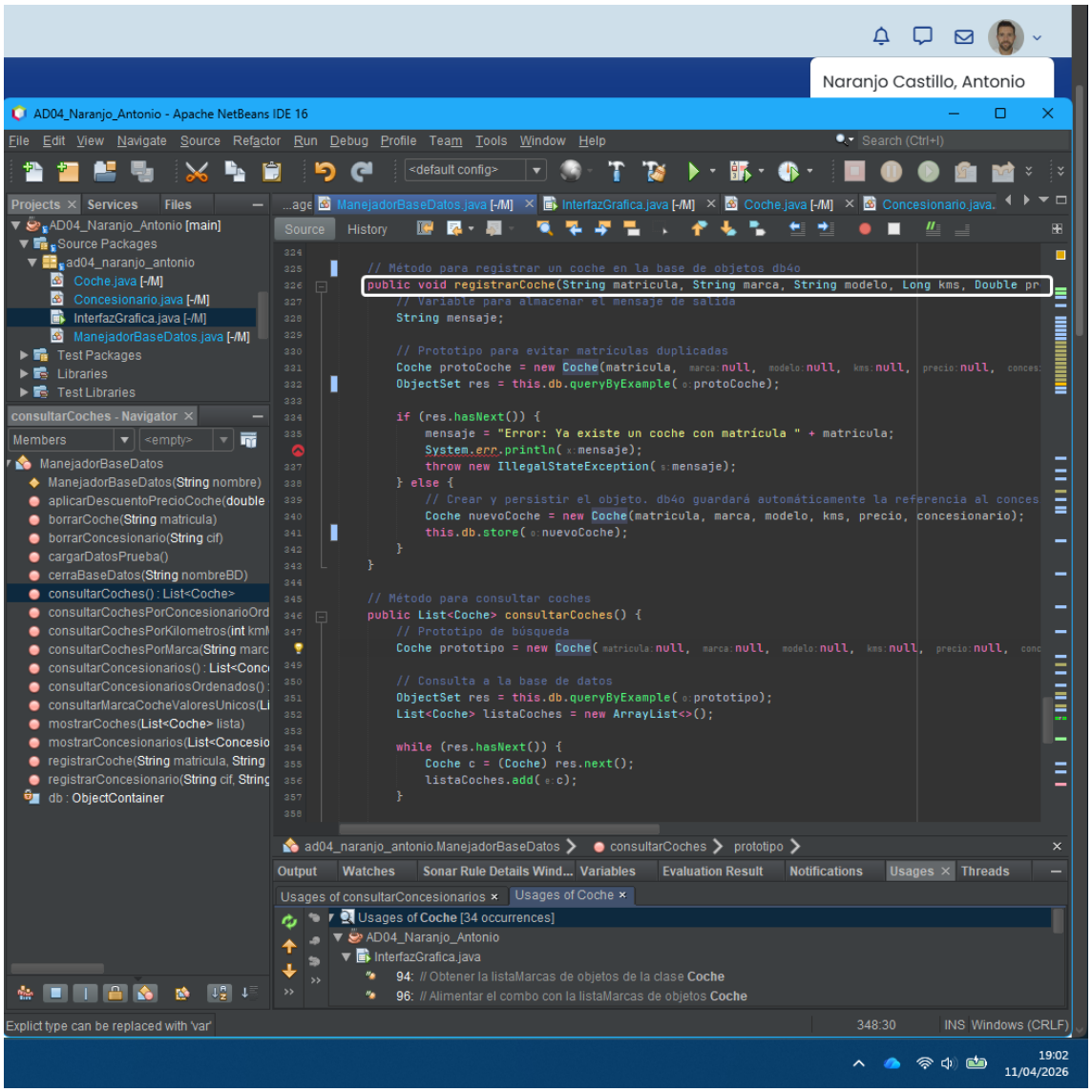
Este método gestiona el registro de nuevos vehículos, integrando una fase de validación de datos antes de proceder a su almacenamiento. Inicialmente, recolecta la información de los campos jTextFieldMatricula, jTextFieldMarca y jTextFieldModelo, asegurando que no estén vacíos y validando que la matrícula cumpla un determinado patrón mediante expresiones regulares. Un punto a tener en cuenta es el tratamiento de datos numéricos, ya que se implementa bloques try-catch internos para convertir los textos de kilómetros a tipo Long y el precio a Double, asegurando el uso de comas como separador decimal mediante el método replace(",", ".").

Tras validar que los valores numéricos son positivos, el método recupera el objeto Concesionario seleccionado en el jComboBoxConcesionario, estableciendo así la relación entre entidades. Con todos los datos verificados, se invoca al método registrarCoche del ManejadorBaseDatos. Finalmente, si la operación tiene éxito, se notifican los cambios al usuario, se actualizan los componentes visuales mediante rellenarComboBorrarCoches y rellenarComboMarcasCoches, y se procede a la limpieza del formulario con limpiarCamposCoche. El control de excepciones captura de forma diferenciada errores de formato, duplicidad de matrículas en db4o o fallos críticos del sistema si los hubiese.

El método registrarCoche es invocado por la interfaz gráfica tras validar los formatos de los campos de texto y recuperar el objeto Concesionario seleccionado por el usuario. Su función: Validar la unicidad de la matrícula mediante un prototipo de búsqueda y persistir el objeto Coche manteniendo la relación de integridad con su concesionario asociado.

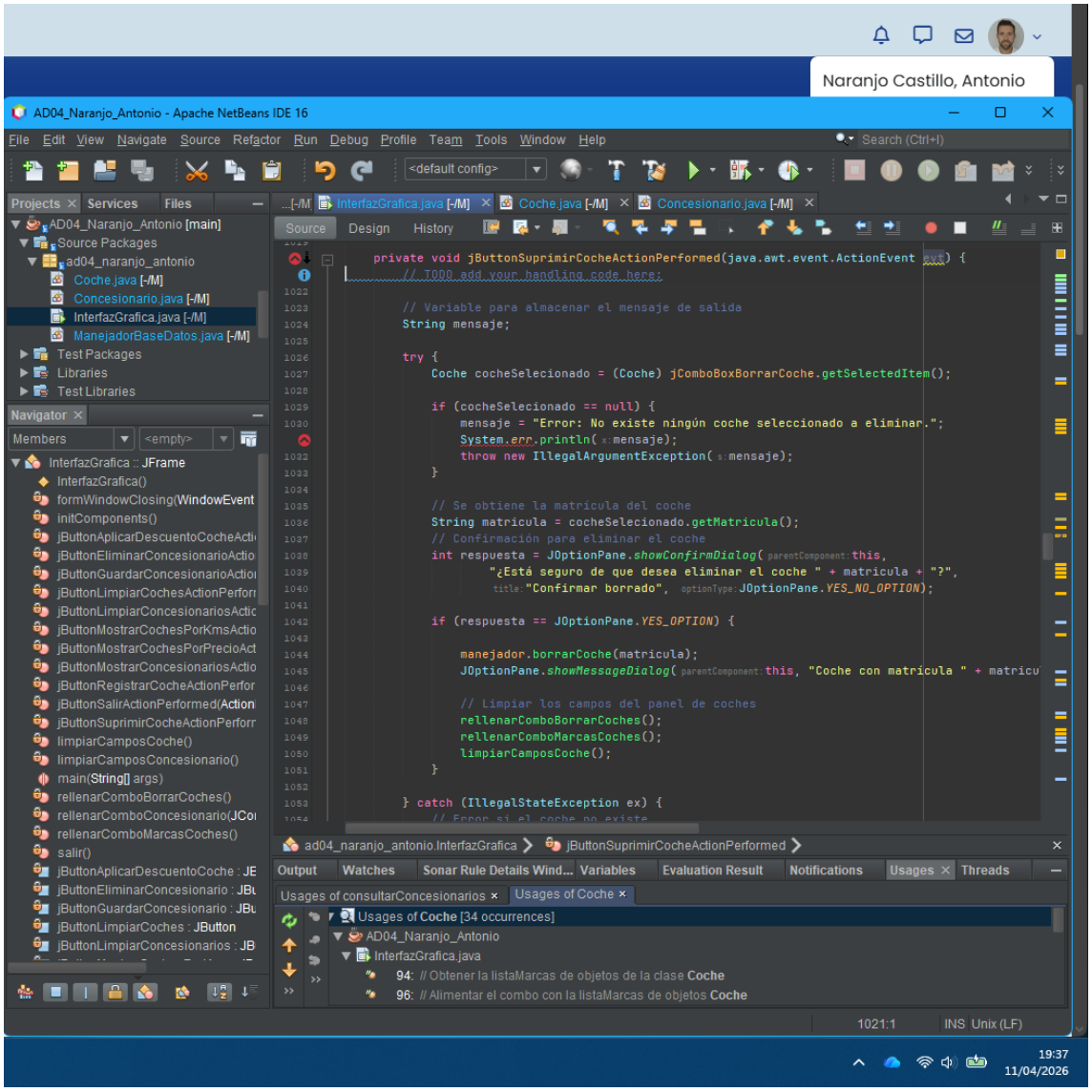
Este método implementa el proceso de alta de vehículos en la base de objetos, asegurando que no se produzcan duplicidades de "claves primarias" lógicas (la matrícula). Para ello, utiliza la técnica Query by Example (QBE) creando un objeto protoCoche que solo contiene la matrícula. Al ejecutar la consulta db.queryByExample(protoCoche), el motor busca coincidencias exactas en el archivo de datos; si el ObjectSet resultante contiene elementos (res.hasNext()), se lanza una excepción IllegalStateException que interrumpe el proceso y notifica el error de duplicidad.

Si la matrícula no existe en la base de datos, se procede a instanciar la clase Coche con todos los parámetros técnicos recibidos. Muy importante la persistencia, al pasar el objeto concesionario (que ya existe en la base de datos) al constructor del coche, db4o no guarda una copia del concesionario, sino una referencia interna al objeto ya persistido. Finalmente, el método store() consolida el nuevo vehículo en el contenedor, vinculándolo permanentemente al concesionario correspondiente.


Naranjo Castillo, Antonio
AD04_Naranjo_Antonio - Apache NetBeans IDE 16
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
Projects Services Files
AD04_Naranjo_Antonio [main]
Source Packages
ad04_naranjo_antonio
Coche.java [-/M]
Concesionario.java [-/M]
InterfazGrafica.java [-/M]
ManejadorBaseDatos.java [-/M]
Test Packages
Libraries
Test Libraries
consultarCoches - Navigator
ManejadorBaseDatos
ManejadorBaseDatos(String nombre)
aplicarDescuentoPrecioCoche(double
borrarCoche(String matricula)
borrarConcesionario(String cif)
cargarDatosPrueba()
cerraBaseDatos(String nombreBD)
consultarCoches() : List<Coche>
consultarCochesPorConcesionarioOrd
consultarCochesPorKilometros(int kml
consultarCochesPorMarca(String marc
consultarConcesionarios() : List<Conc
consultarConcesionariosOrdenados() :
consultarMarcaCocheValoresUnicos(Li
mostrarCoches(List<Coche> lista)
mostrarConcesionarios(List<Concesio
registrarCoche(String matricula, String
registrarConcesionario(String cif, String
db : ObjectContainer
Source History
// Método para registrar un coche en la base de objetos db4o
public void registrarCoche(String matricula, String marca, String modelo, Long kms, Double pr
String mensaje;
// Prototipo para evitar matriculas duplicadas
Coche protoCoche = new Coche(matricula,
ObjectSet res = this.db.queryByExample(
if (res.hasNext()) {
mensaje = "Error: Ya existe un coche con matricula " + matricula;
System.err.println(
throw new IllegalStateException(
} else {
// Crear y persistir el objeto. db4o guardará automáticamente la referencia al conces
Coche nuevoCoche = new Coche(matricula, marca, modelo, kms, precio, concesionario);
this.db.store(
// Método para consultar coches
public List<Coche> consultarCoches() {
// Prototipo de búsqueda
Coche prototipo = new Coche(
// Consulta a la base de datos
ObjectSet res = this.db.queryByExample(
List<Coche> listaCoches = new ArrayList<>();
while (res.hasNext()) {
Coche c = (Coche) res.next();
listaCoches.add(
ad04_naranjo_antonio.ManejadorBaseDatos consultarCoches prototipo
Output Watches Sonar Rule Details Wind... Variables Evaluation Result Notifications Usages Threads
Usages of consultarConcesionarios Usages of Coche
Usages of Coche [34 occurrences]
AD04_Naranjo_Antonio
InterfazGrafica.java
94: // Obtener la listaMarcas de objetos de la clase Coche
96: // Alimentar el combo con la listaMarcas de objetos Coche
Explict type can be replaced with 'var'
348:30 INS Windows (CRLF)
19:02
11/04/2026

1.4. Eliminar coches de la base de datos.

Se ejecuta el método jButtonSuprimirCocheActionPerformed al pulsar el botón "Suprimir" (o Eliminar) dentro del panel de gestión de bajas de vehículos. Gestionando la eliminación de un vehículo seleccionado por el usuario, solicitando confirmación previa y actualizando los componentes visuales relacionados con el inventario de coches.



Este método controla el flujo de baja de vehículos en la interfaz, asegurando que la operación se realice de forma segura y controlada. El proceso comienza recuperando el objeto Coche seleccionado en el componente jComboBoxBorrarCoche; si no hay ninguna selección activa, se lanza una excepción de tipo IllegalArgumentException. Una vez identificado el vehículo, se extrae su matrícula para personalizar un cuadro de diálogo de confirmación mediante JOptionPane.showConfirmDialog. Esta medida de seguridad impide eliminaciones accidentales al requerir una validación del usuario.

Tras obtener la conformidad, se invoca al método borrarCoche(matricula) del ManejadorBaseDatos para ejecutar la eliminación en la base de objetos. Para mantener la consistencia de la vista, el método refresca inmediatamente los desplegables de la

aplicación mediante rellenarComboBorrarCoches y rellenarComboMarcasCoches, además de resetear el formulario con limpiarCamposCoche. El sistema de control de errores captura excepciones de estado legal, argumentos no válidos o fallos genéricos, informando al usuario mediante ventanas emergentes con el icono de advertencia o error correspondiente.

El método borrarCoche es invocado por la interfaz gráfica tras la confirmación del usuario, recibiendo como parámetro la matrícula del vehículo a eliminar. Localiza de forma unívoca el objeto Coche en la base de datos db4o y ejecutar su borrado físico del contenedor de objetos.

```
// Método para borrar un coche de la base de objetos db4o
public void borrarCoche(String matricula) {
    // Variable para almacenar el mensaje de salida
    String mensaje;

    // Buscar el coche por su matricula, se crea un prototipo de consulta
    Coche proto = new Coche(matricula, marca:null, modelo:null, kms:null, precio:null, concesionario
    ObjectSet res = this.db.queryByExample( o:proto);

    if (res.hasNext()) {
        // Si existe, se recupera el objeto y se elimina
        Coche c = (Coche) res.next();
        this.db.delete( o:c);
        mensaje = "Coche con matricula " + matricula + " eliminado de la base de objetos.";
        System.out.println( x:mensaje);
    } else {
        // Si no existe, se lanza excepción
        mensaje = "No se encontró ningún coche con la matricula: " + matricula;
        System.err.println( x:mensaje);
        throw new IllegalStateException( s:mensaje);
    }
}

// Método para registrar un coche en la base de objetos db4o
public void registrarCoche(String matricula, String marca, String modelo, Long kms, Double pr
    // Variable para almacenar el mensaje de salida
    String mensaje;

    // Prototipo para evitar matriculas duplicadas
    Coche protoCoche = new Coche(matricula, marca:null, modelo:null, kms:null, precio:null, conces:
    ObjectSet res = this.db.queryByExample( o:protoCoche);

    if (res.hasNext()) {
        mensaje = "Error: Ya existe un coche con matricula " + matricula;
```

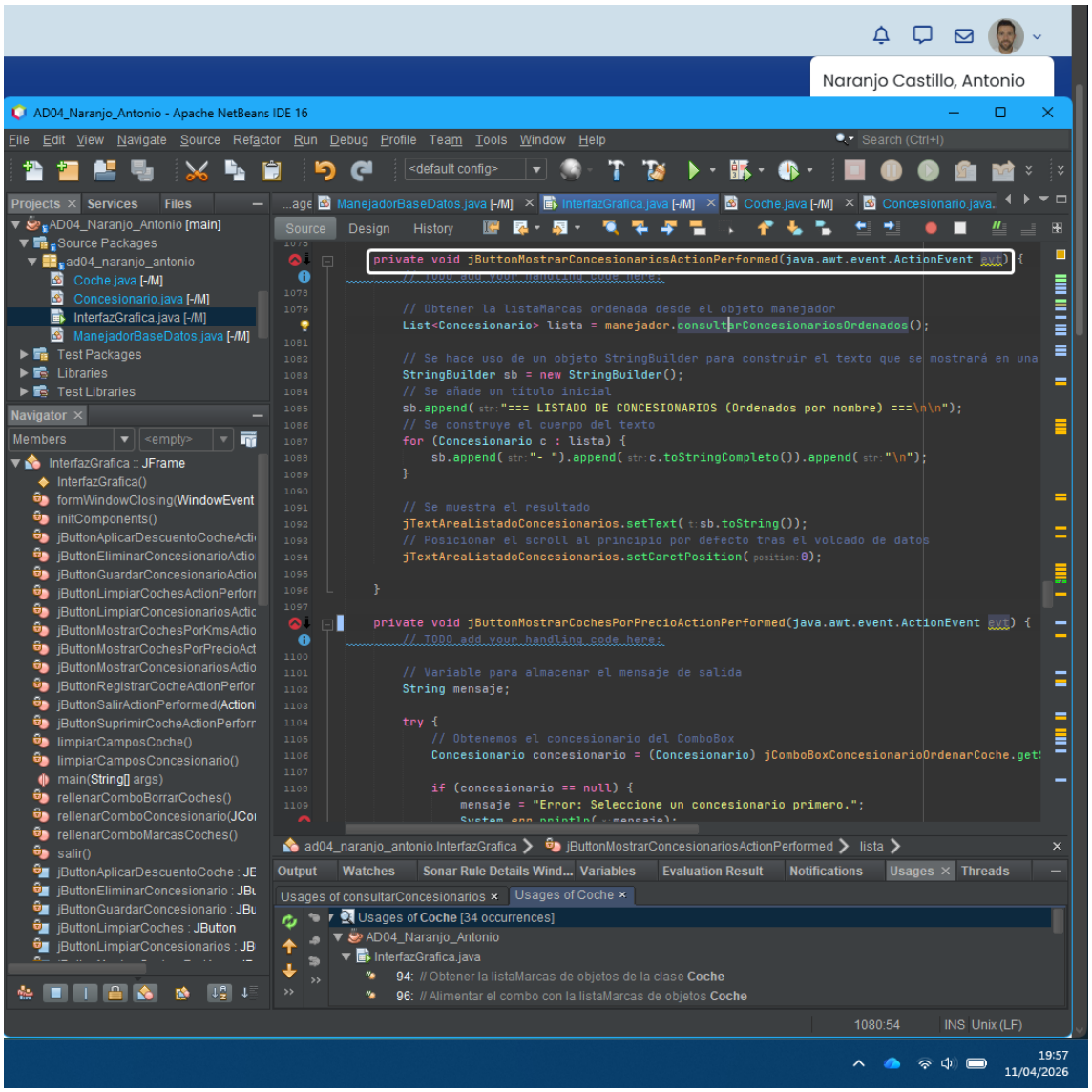
Este método implementa la lógica de eliminación de vehículos en el motor de persistencia. El proceso se inicia mediante la creación de un objeto prototipo de la clase Coche, donde se asigna la matrícula recibida y se establecen el resto de atributos como null. Utilizando la técnica Query by Example (QBE) a través de db.queryByExample(proto), se busca el registro correspondiente en la base de datos.

Si la consulta devuelve un resultado positivo (res.hasNext()), se recupera la instancia del objeto mediante el método next() y se procede a su eliminación definitiva invocando a this.db.delete(c).

A diferencia de la eliminación de concesionarios, este método no requiere comprobaciones de integridad referencial adicionales, ya que el coche es objeto simple en el modelo de datos. En caso de que la matrícula no se encuentre en el sistema, el método lanza una excepción IllegalStateException y registra el error en la consola.

1.5. Mostrar todos los concesionarios ordenados por nombre.

Se ejecuta el método jButtonMostrarConcesionariosActionPerformed para mostrar los concesionarios ordenados por nombre pulsando el botón "Mostrar" en el panel de consultas de concesionarios, recuperando la colección de concesionarios ordenados alfabéticamente, construir una cadena de texto formateada y volcarla sobre un componente de texto de la interfaz para su lectura.

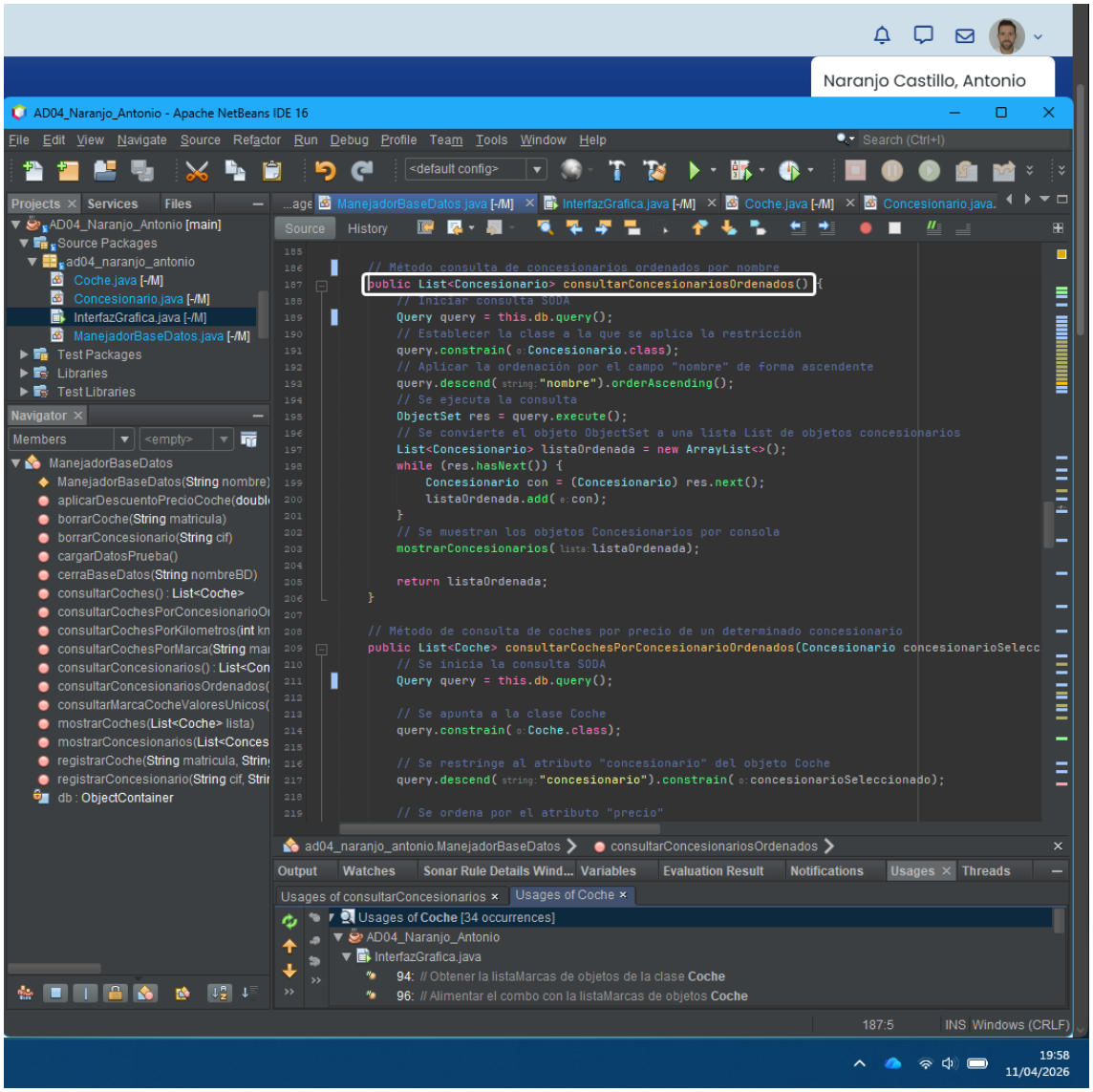


Este método gestiona la generación del listado completo de concesionarios mediante un proceso de recuperación y formateo de datos. En primer lugar, obtiene la colección de objetos invocando al método consultarConcesionariosOrdenados() de la clase ManejadorBaseDatos, lo que asegura que la información se reciba ya clasificada. Para optimizar el rendimiento y la gestión de memoria durante la construcción de la salida, el método utiliza un objeto StringBuilder en lugar de concatenaciones de cadenas convencionales.

El flujo recorre la lista obtenida mediante un bucle for-each, añadiendo a cada línea el resultado del método toStringCompleto() de la instancia de Concesionario. Una vez

finalizada la construcción del texto, el contenido se vuelca en el componente jTextAreaListadoConcesionarios mediante el método setText(). Finalmente, se ejecuta la instrucción setCaretPosition(0), mejora de usabilidad que posiciona el scroll de la barra de desplazamiento al inicio del documento, garantizando que el usuario vea el listado desde el primer registro tras la carga.

Con el método consultarConcesionariosOrdenados se recupera la colección completa de objetos Concesionario aplicando un criterio de ordenación alfabético ascendente basado en el atributo "nombre".



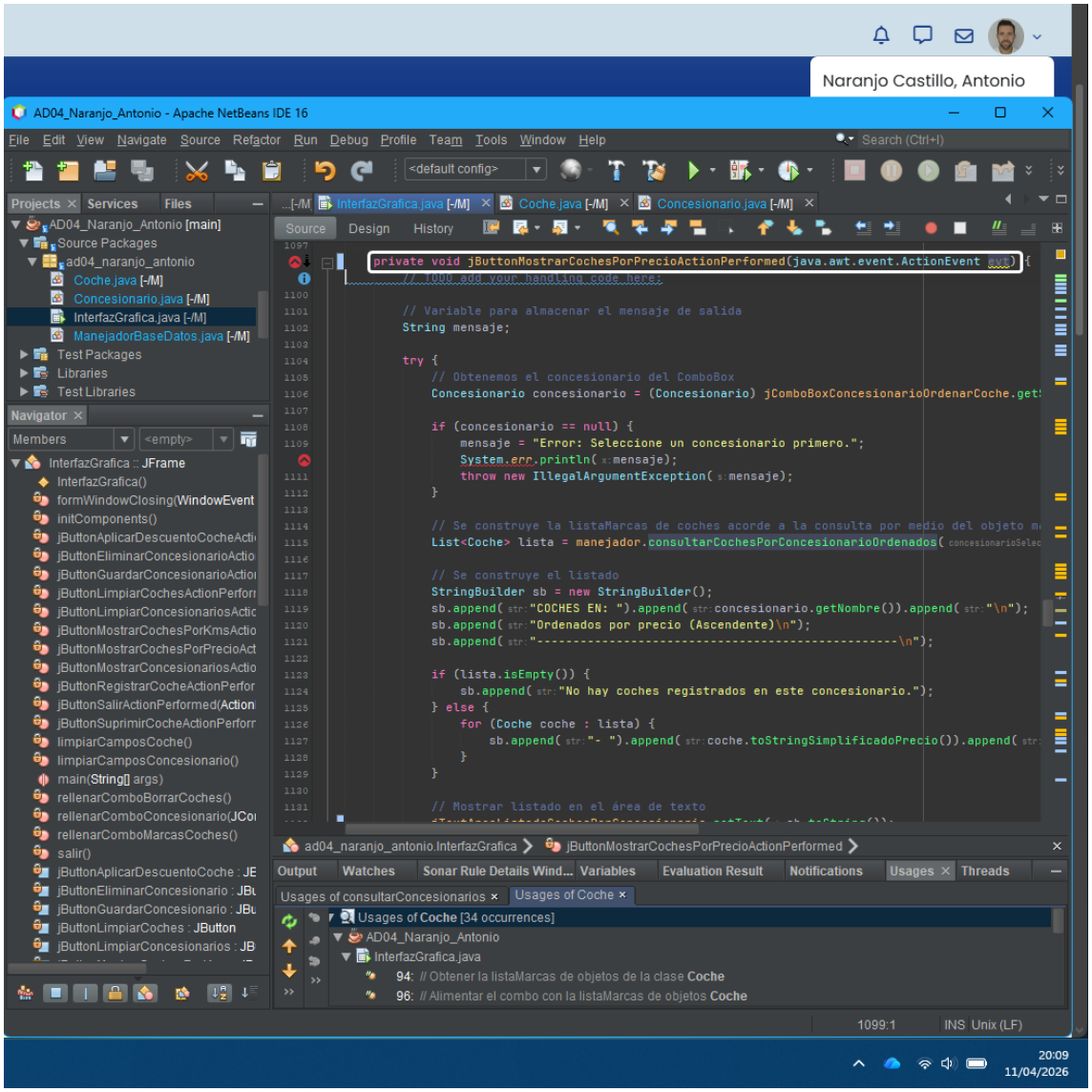
Este método implementa una consulta avanzada mediante la interfaz SODA. A diferencia de las consultas por ejemplo (QBE), SODA permite definir un grafo de navegación sobre los objetos. El proceso comienza instanciando un objeto Query y aplicando una restricción de clase mediante constrain(Concesionario.class). La potencia de este método reside en la instrucción descend("nombre").orderAscending(),

que indica al motor de la base de datos que debe descender al atributo específico del objeto y clasificar los resultados de forma ascendente antes de devolverlos.

Una vez ejecutada la consulta con query.execute(), el ObjectSet resultante se procesa mediante un bucle while. En cada iteración, se extrae el objeto, se realiza el casting a la clase Concesionario y se añade a un ArrayList. Antes de retornar la lista se llama al método auxiliar mostrarConcesionarios(listaOrdenada) para registrar la salida en la consola del sistema, permitiendo verificar que la ordenación se ha realizado correctamente.

1.6. Mostrar todos los coches de un concesionario indicado por el usuario, ordenados por precio.

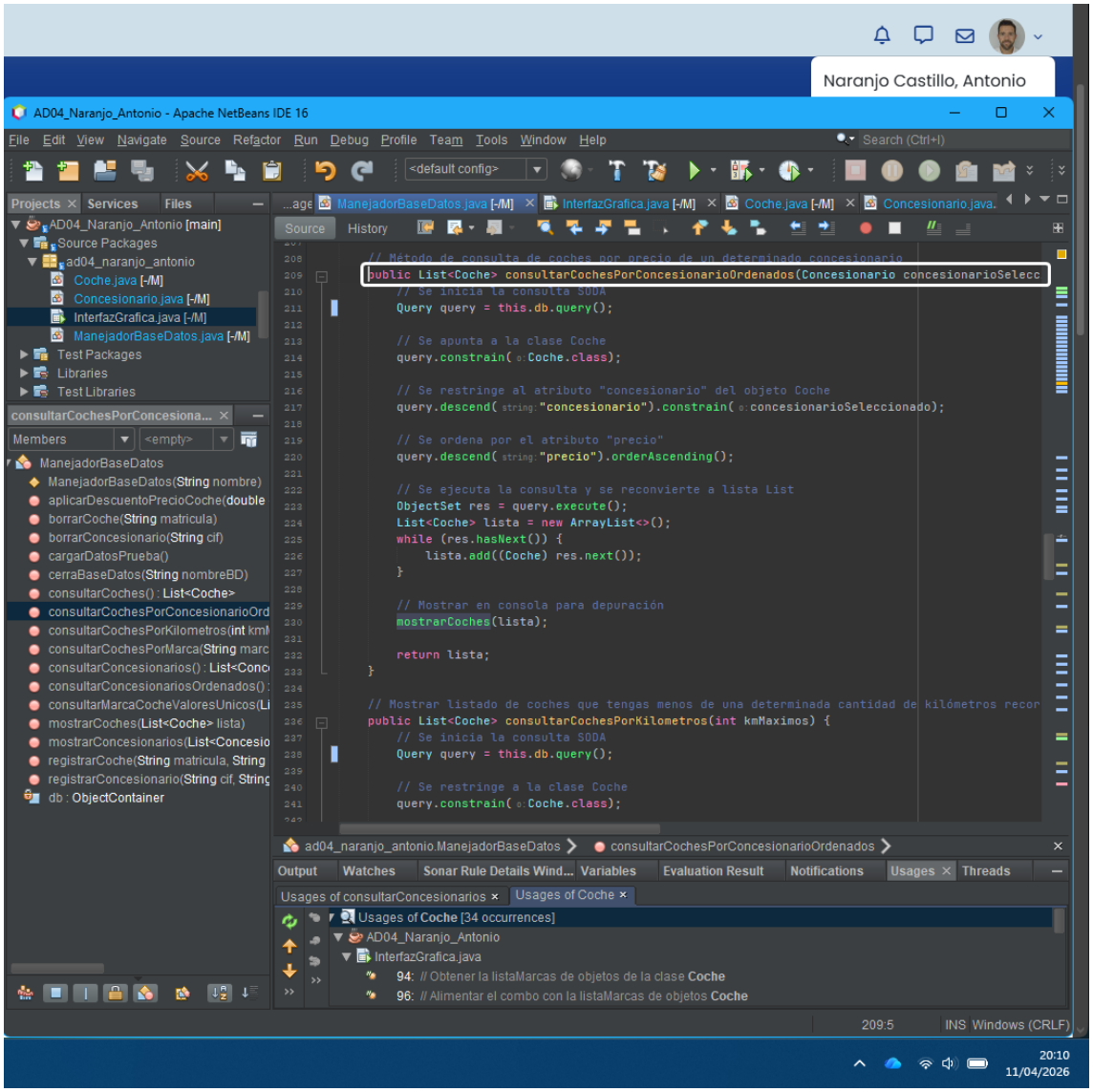
Con el método jButtonMostrarCochesPorPrecioActionPerformed, que se activa al pulsar el botón “Mostrar” dentro del panel de listado de coches por concesionario, se filtran los vehículos pertenecientes a un concesionario específico, recuperarlos ordenados por su precio y mostrando los resultados de forma tabulada en la interfaz gráfica.



Este método implementa una lógica de consulta parametrizada que depende de la selección del usuario. El flujo comienza recuperando el objeto Concesionario del componente jComboBoxConcesionarioOrdenarCoche. Si no hay una selección válida, el sistema lanza una excepción IllegalArgumentException, que es capturada para mostrar un mensaje de advertencia al usuario mediante un JOptionPane. Mediante el método consultarCochesPorConcesionarioOrdenados(concesionario) una vez validada la selección, se solicita al ManejadorBaseDatos la lista de coches asociada a dicha instancia.

Para la construcción del informe visual, se emplea un StringBuilder. El método genera una cabecera dinámica que incluye el nombre del concesionario seleccionado y procede a iterar sobre la lista recuperada. Durante el bucle, se utiliza el método especializado de la entidad toStringSimplificadoPrecio(), que presenta la marca, el modelo, la matrícula y el PVP formateado. Finalmente, el contenido se vuelca en el componente jTextAreaListadoCochesPorConcesionario, asegurando que el scroll se posicione al inicio para una lectura cómoda mejorando la usabilidad de la aplicación. El diseño del código garantiza que, si no existen coches para ese concesionario, se informe explícitamente al usuario en lugar de dejar el área de texto vacía.

Con el método consultarCochesPorConcesionarioOrdenados se filtra la colección de objetos Coche basándose en su asociación con un Concesionario determinado y devuelve los resultados clasificados por precio de menor a mayor.



Este método utiliza una consulta SODA para realizar una búsqueda avanzada por asociación. El proceso comienza instanciando un objeto Query y restringiéndolo a la

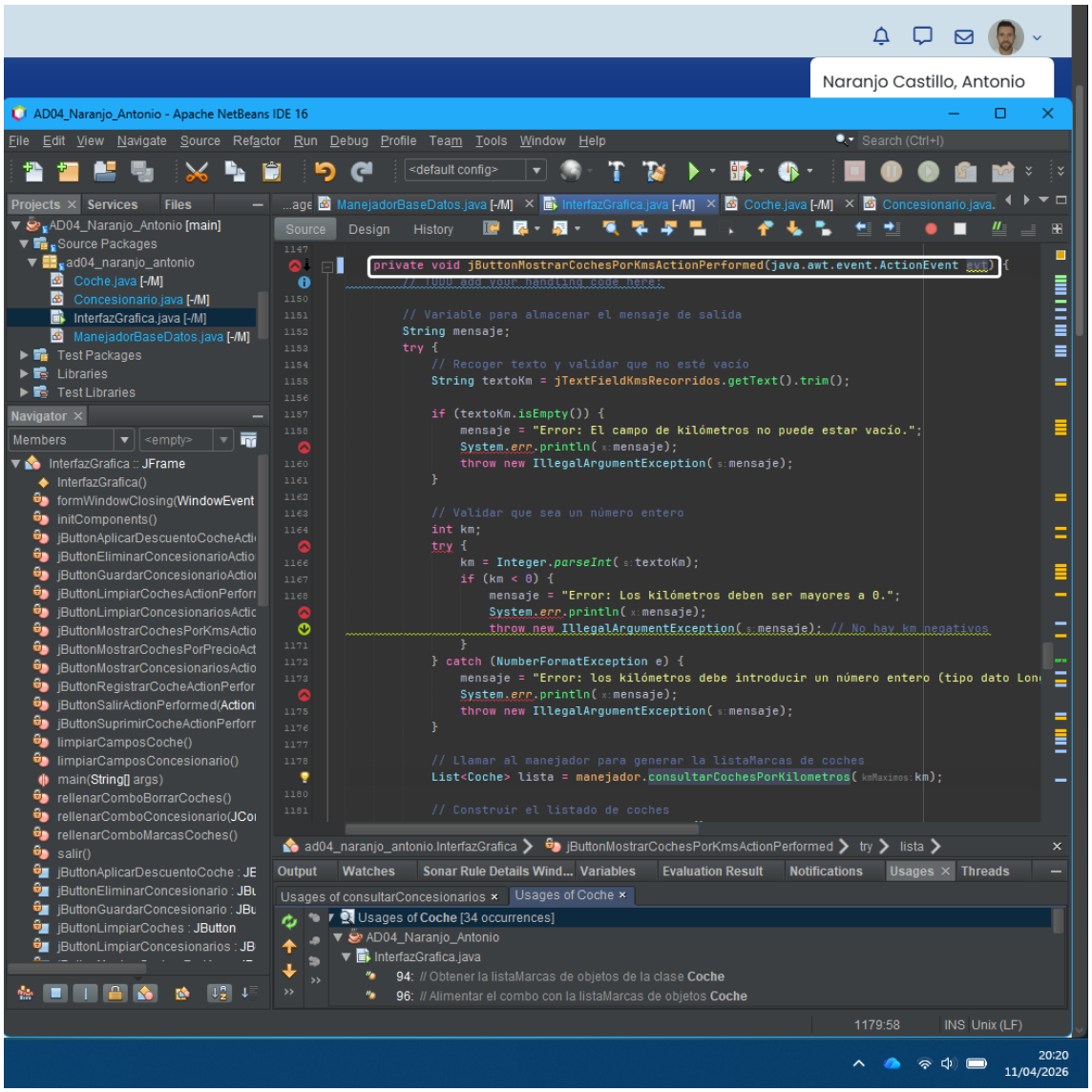
clase Coche. Lo más destacado de esta implementación es el uso de query.descend("concesionario").constrain(concesionarioSeleccionado). En lugar de filtrar por un ID o una cadena de texto, SODA permite filtrar directamente por la instancia del objeto, comparando las referencias de memoria dentro de la base de datos.

Además del filtro, mediante query.descend("precio").orderAscending() se aplica una regla de ordenación, lo que garantiza que los objetos recuperados sigan un orden numérico ascendente. Tras la ejecución, el ObjectSet se vuelca en un ArrayList mediante un iterador. El motor de db4o procesa el filtrado y la ordenación de forma nativa reduciendo la carga de procesamiento en la memoria principal.

1.7. Mostrar todos los coches que tengan menos de un determinado número de kilómetros (el número de kilómetros lo debe facilitar el usuario).

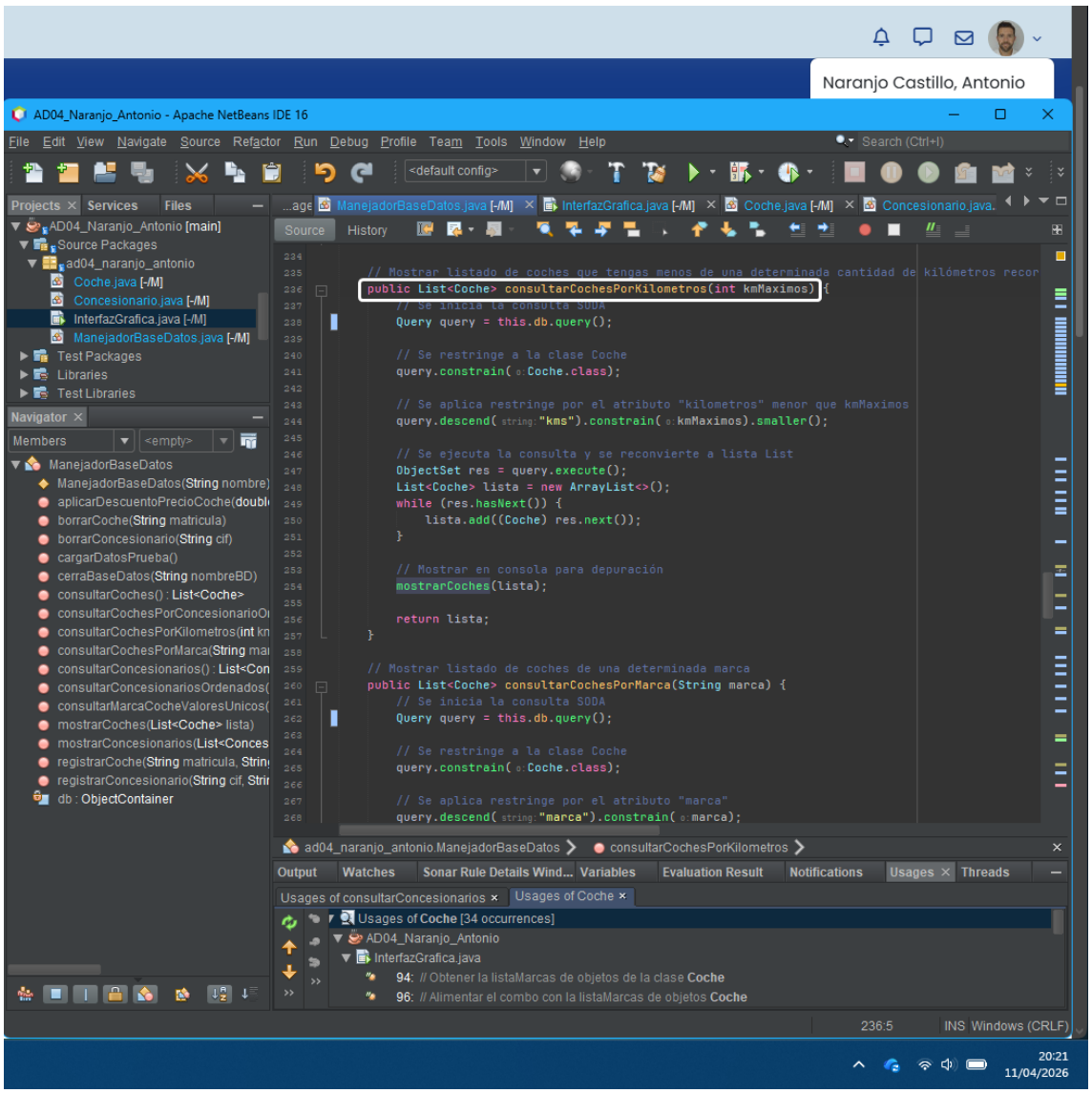
Se ejecuta al pulsar el botón “mostrar” en el panel de filtrado por kilómetros el método jButtonMostrarCochesPorKmsActionPerformed que se encarga de validar la entrada numérica del usuario, solicitar al manejador los vehículos que no superen dicho kilometraje y presentar los resultados formateados en la interfaz.

Este método implementa una lógica de consulta dinámica basada en un dato numérico proporcionado por el usuario. El proceso se inicia extrayendo el texto del campo jTextFieldKmsRecorridos. El método aplica una validación en dos fases: primero comprueba que el campo no esté vacío y, posteriormente, intenta convertir la cadena a un tipo de dato entero (int). Mediante un bloque try-catch interno para NumberFormatException, se asegura de que la entrada sea un número válido y no negativo, lanzando una IllegalArgumentException personalizada si los datos son incoherentes.



Tras superar la validación, se invoca al método consultarCochesPorKilometros(km) de la capa de persistencia. Para la salida de datos, se utiliza nuevamente la clase StringBuilder, construyendo una cabecera informativa que indica el criterio aplicado. El listado recorre la colección recuperada empleando el método de la entidad toStringSimplificadoKms(), que resalta el kilometraje y el CIF del concesionario asociado. Finalmente, el texto se vuelca en jTextAreaListadoCochesPorKms y se resetea la posición del scroll. El control de excepciones captura cualquier error de validación o fallo en la consulta para informar al usuario mediante cuadros de diálogo de advertencia.

El método consultarCochesPorKilometros recupera de la base de objetos todos los registros de la clase Coche cuyo atributo de kilometraje sea inferior al valor entero proporcionado como parámetro.



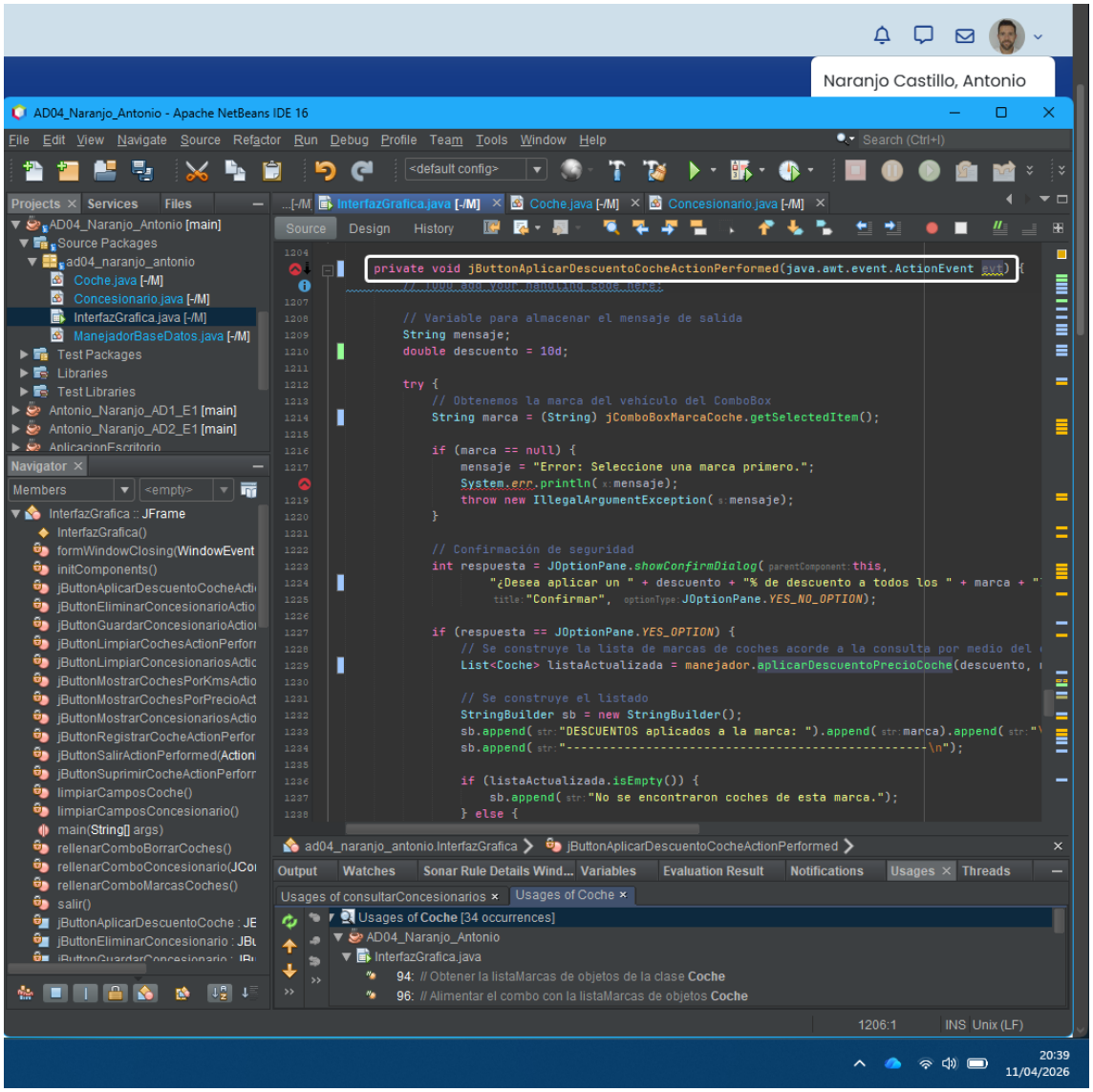
Este método implementa una consulta SODA. El proceso se inicia configurando un objeto Query que apunta a la clase Coche. Se concatenan las restricciones tal que

query.descend("kms").constrain(kmMaximos).smaller(). Esta sintaxis le indica al motor de db4o que debe evaluar el campo "kms" y aplicar un filtro comparativo de "menor estricto" respecto al valor kmMaximos.

A diferencia de las consultas QBE (Query by Example), que se basan en la igualdad, esta consulta SODA permite realizar búsquedas por criterios de magnitud. Una vez ejecutado el método execute(), el conjunto de resultados (ObjectSet) se transforma en un ArrayList de Java. El método también incluye una llamada a mostrarCoches(lista) para facilitar el seguimiento de la consulta a través de la consola, pudiéndose comprobar si los objetos recuperados cumplen exactamente con el criterio de filtrado.

1.8. Rebajar el precio en un 10% de todos los coches de una determinada marca (la marca la debe introducir el usuario).

El método jButtonAplicarDescuentoCocheActionPerformed se ejecuta al pulsar el botón "Aplicar Descuento" en el panel de descuentos. Trata de validar la selección de una marca, solicitar confirmación al usuario para una operación crítica y reflejar en la interfaz el resultado de la actualización de precios tras aplicar un descuento porcentual.

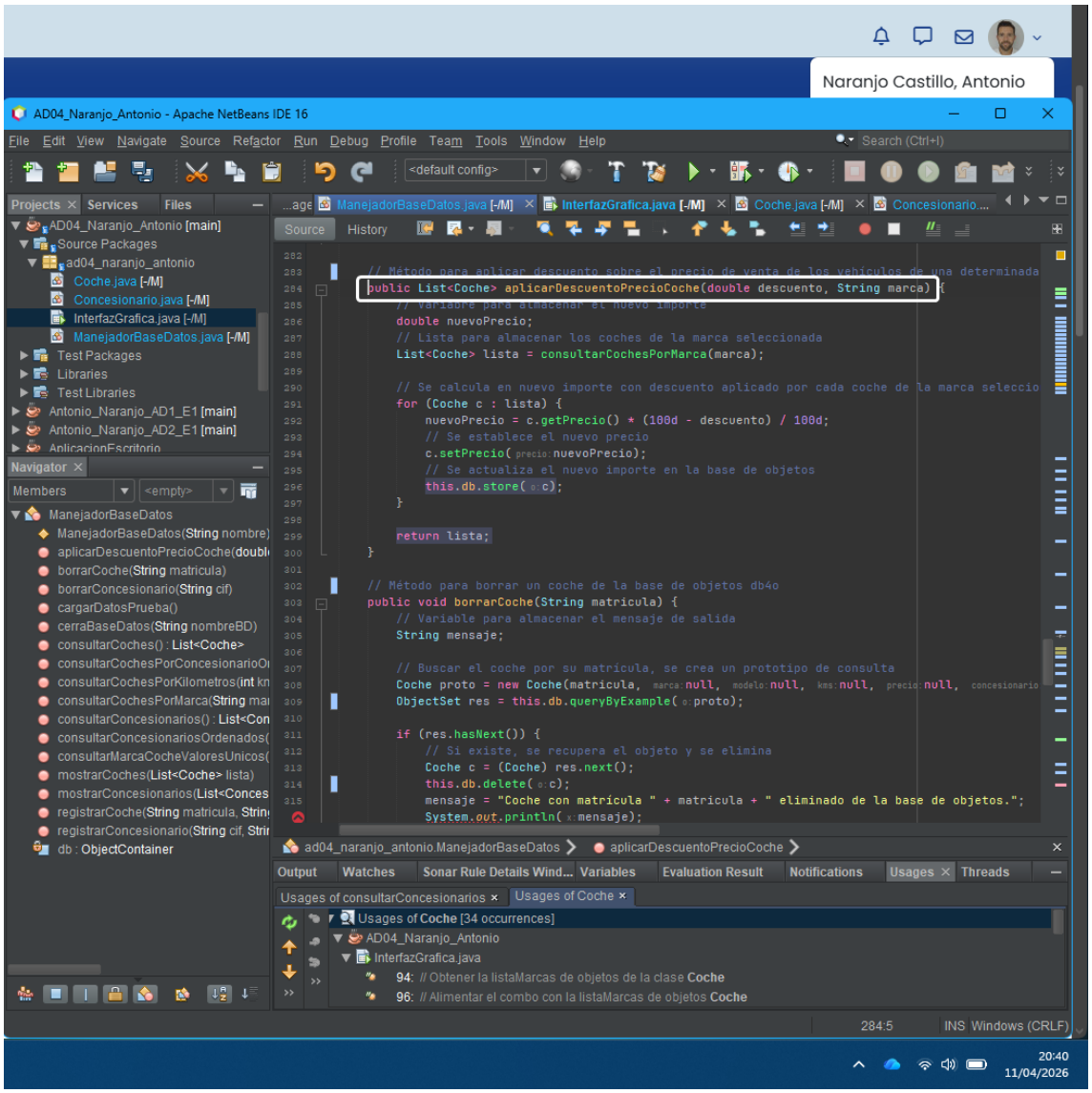


Este método realiza el mantenimiento sobre la base de datos. Comienza recuperando la marca seleccionada desde jComboBoxMarcaCoche. Dado que una modificación de precios puede alterar los datos registrados en la base de datos, el método implementa una confirmación de seguridad mediante JOptionPane.showConfirmDialog, informando explícitamente que se aplicará un 10% de descuento. Solo si el usuario responde afirmativamente (YES_OPTION), se procede a llamar al método manejador.aplicarDescuentoPrecioCoche(descuento, marca).

La respuesta del manejador es una lista de objetos Coche ya actualizados. El método utiliza un StringBuilder para generar un informe detallado de la operación, recorriendo

los coches afectados y mediante toStringSimplificadoPrecio() muestra su nuevo PVP. Finalmente, en el área de texto jTextAreaListadoCochesPorMarca se vuelca el listado. El manejo de excepciones asegura que, si no se selecciona una marca u ocurre un error durante la actualización en el motor db4o, el usuario sea notificado mediante mensajes de advertencia o error.

El método aplicarDescuentoPrecioCocheAcción cumple la función de localizar todos los vehículos de una marca determinada, recalcular su precio aplicando un porcentaje de descuento y persistir los cambios de forma permanente en la base de datos.

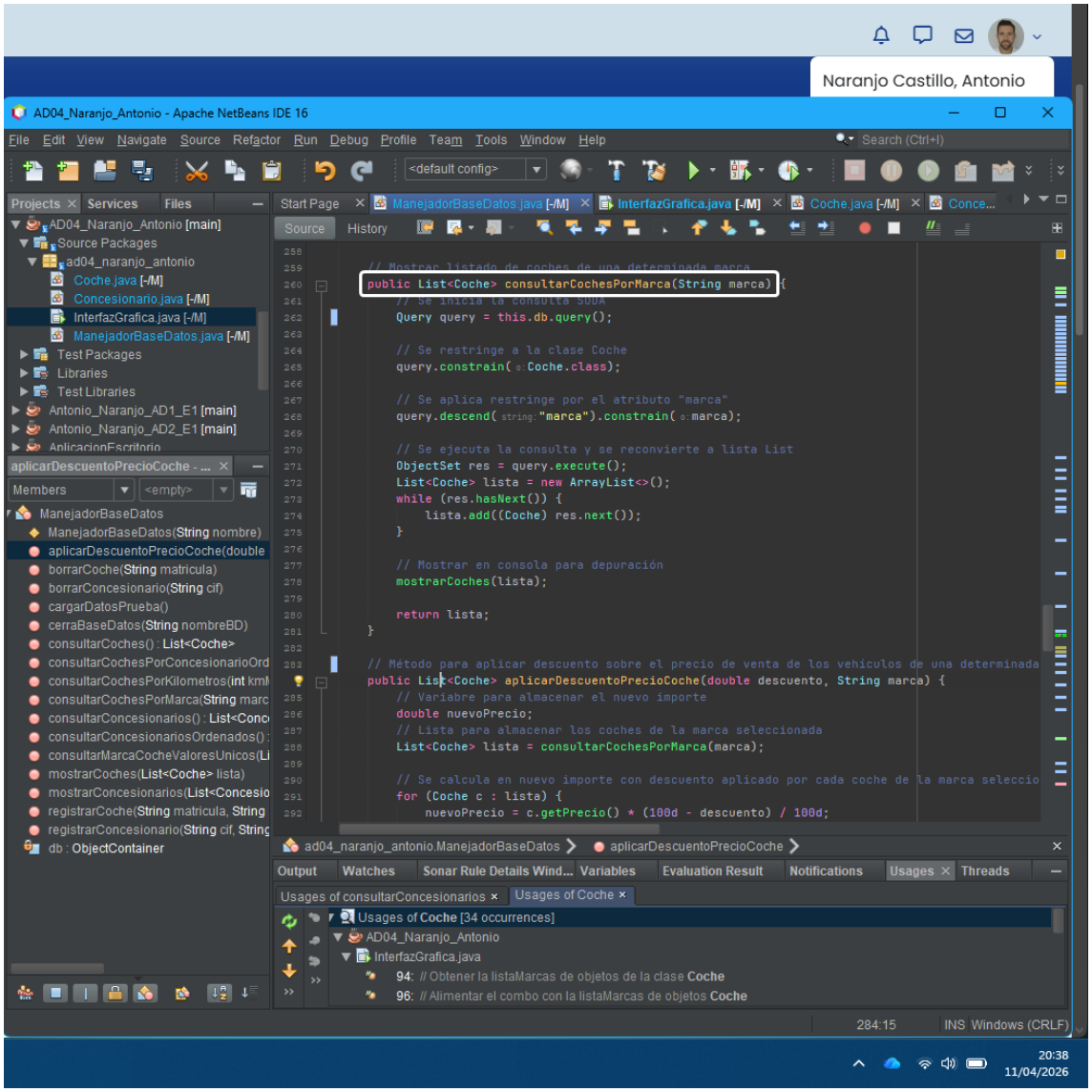


Este método implementa una lógica de actualización en bloque, en primer lugar, utiliza el método auxiliar consultarCochesPorMarca(marca) para recuperar una lista con todos los objetos Coche que coinciden con el criterio. Una vez obtenida la colección, el método itera sobre cada instancia mediante un bucle for-each.Dentro del bucle, se realiza el cálculo aritmético del nuevo importe.

Tras recalcular el valor, se actualiza el estado del objeto mediante el método c.setPrecio(nuevoPrecio). El paso importante ocurre con la instrucción

this.db.store(c): en db4o, si el objeto ya existe en la base de datos (identificado por su referencia interna), el método store no crea un duplicado, sino que sobrescribe los atributos modificados. Finalmente, el método retorna la lista de objetos actualizados para que la interfaz pueda mostrar los nuevos precios al usuario.

El método consultarCochesPorMarca localiza y devuelve todos los objetos de la clase Coche cuyo atributo "marca" coincida exactamente con la cadena de texto proporcionada.



Este método implementa una consulta filtrada mediante una consulta SODA. El proceso se inicia con la creación de una instancia Query y la definición de su alcance mediante constrain(Coche.class). La clave del filtrado reside en el método query.descend("marca").constrain(marca), que indica al motor de db4o que debe buscar en el campo específico de la clase y devolver únicamente aquellos objetos que tengan una coincidencia exacta con el parámetro recibido.